Introduction to software engineering project

Deliverable three

Contents

The Noun Technique 3

Methodology 3

Raw Noun List 3

Classification 3

Final Domain Class List 5

UML Class Diagram 6

UML Class Diagram – Explanation 7

Modeling Assumptions 7

State Machine Diagram 7

Brief Explanation 8

Use Cases, CRUD & State Machines – Explanation 9

Modeling Steps and Assumptions 9

• Step 4 – Findings: 10

• Step 5 – Refinements: 10

Traceability Overview 10

State Machine Diagrams 11

• State Machine #1 – Bill Lifecycle: 11

• State Machine #2 – Appointment Lifecycle: 12

State Machine #2 – Appointment Lifecycle (Explanation) 13

Use Case Explanations – Selected Complex Use Cases 13

UC-01 Book Appointment – Detailed Textual Flow 13

UC-06 Cancel Appointment – Detailed Textual Flow 16

UC-07 Make Payment – Detailed Textual Flow 17

Activity Diagrams – Explanations for Report 18

UC-01 Book Appointment – Activity Diagram Explanation 18

UC-06 Cancel Appointment – Activity Diagram Explanation 19

UC-07 Make Payment – Activity Diagram Explanation 20

Inputs for SSD Design 21

System Sequence Diagrams – Design Approach 22

UC-02 Register patient information – SSD 23

UC-03 Manage doctor schedule – SSD 24

UC-04 Confirm appointment slot – SSD 25
UC-05 Complete patient treatments – SSD 26
UC-08 Authorize patient discharge – SSD 26
UC-09 Send appointment reminders – SSD 27
UC-10 Generate weekly report schedule – SSD 29
UC-11 Generate monthly bill – SSD 30
UC-12 Clean up expired slots – SSD 31
UC-T1 Handle payment failure – SSD 31
UC-T2 Update slot availability – SSD 33
UC-T3 Synchronize patient information – SSD 34
Alternative and Exception Scenarios in SSDs 34
Workload 35
Jira Management 39

Introduction to software engineering project

Deliverable three

The Noun Technique

Methodology

To identify the core domain classes for the Hospital System, we utilized the Noun Technique. This process involved parsing the system requirements and case study documents to extract all nouns. These nouns were then filtered and categorized to distinguish between potential classes, attributes of classes, synonyms, and irrelevant concepts. This ensures that our domain model accurately reflects the business requirements without unnecessary clutter.

Raw Noun List

Below is the extracted list of nouns found in the case study.

Raw Nouns		
Patient	Doctor	Appointment
Nurse	Schedule	Room
Medicine	Prescription	Bill
Payment	Receptionist	Surgery
Ward	Bed	Diagnosis
User	System	Report
Date	Time	ID Number

Classification

We analyzed the raw nouns to determine their role in the system.

Noun	Classification	Rationale / Decision
Patient	Class	Core entity receiving care.
Doctor	Class	Core entity providing care.
Appointment	Class	Event linking Patient and Doctor; has its own lifecycle.
Nurse	Class	Medical staff supporting Doctors; modeled as a specialization of Staff in the class diagram.
Schedule	Attribute	Represented as a timetable attribute/collection inside Doctor, not as a separate persistent class.
Room	Class	Physical location for admitted patients or consultations; kept as a domain class.
Medicine	Class	Inventory item tracked by the system.

Introduction to software engineering project

Deliverable three

Prescription	Class	Order that groups one or more Medicine items for a patient.
Bill	Class	Financial record for services rendered; used by the billing process.
Payment	Operation	Modeled as operations/status changes on Bill (e.g., pay(), paymentStatus) rather than a separate persistent class.
Receptionist	Actor	Human role interacting with the system UI; not stored as a domain class.
Surgery	Class (specialization, not in final list)	Specific type of treatment; considered a specialization of consultation/treatment but omitted from the simplified final domain class list.
Ward	Attribute / Value	Treated as a value of Room.type (e.g., ICU/Ward), not a separate class.
Bed	Attribute	Modeled as an attribute of Room (e.g., numberOfBeds), not as a separate class.
Diagnosis	Attribute	Stored as attribute(s) of Consultation / MedicalRecord, not its own class.
User	Actor / Abstract concept	Represents human roles (patient, doctor, receptionist, admin); realized as specific actors or as Person/Staff, not kept as a separate persistent class.
System	Irrelevant	Refers to the software itself, not a domain object.
Report	Irrelevant / Derived artifact	Generated reporting output (e.g., monthly reports); not a core persistent domain class.
Date	Attribute	Part of time-stamped attributes for Appointment and other events.
Time	Attribute	Part of time-stamped attributes for Appointment and other events.
ID Number	Attribute	Identifier attribute for Patient/Doctor (e.g., patientID, empID).

- For CRUD and SRS we focus on the seven core persistent domain classes (Patient, Doctor, Appointment, MedicalRecord, Prescription, Room, Bill). Additional supporting classes such as Person, Staff, Nurse, Consultation and Medicine are introduced later in the UML class diagram as refinements/generalizations of these core concepts.

Final Domain Class List

The following table defines the finalized classes derived from the analysis, including their primary attributes.

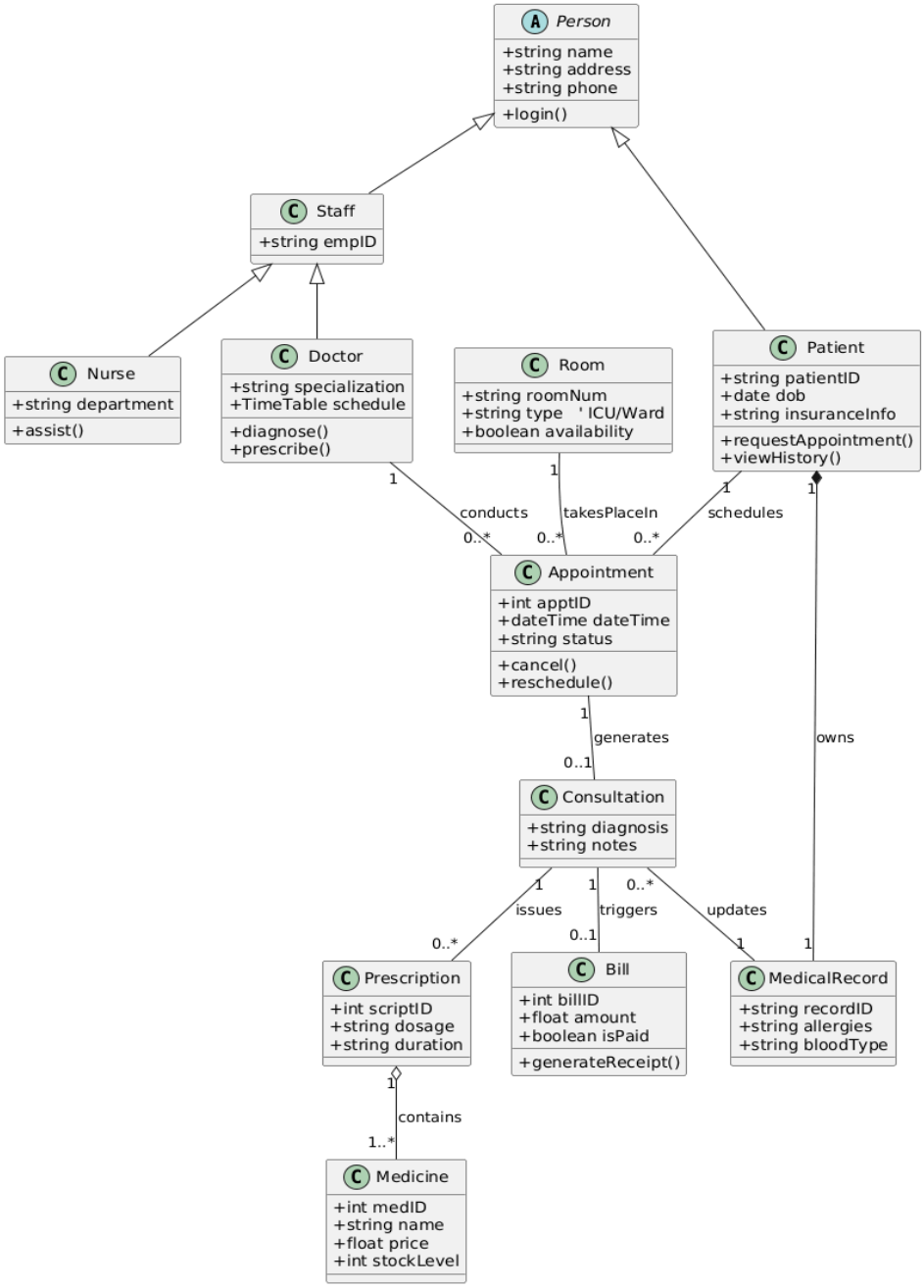
Class Name	Description	Key Attributes
Patient	An individual seeking medical care.	patientID, name, dob, insuranceDetails, history
Doctor	Medical staff conducting consultations.	empID, specialization, schedule, name
Appointment	A scheduled meeting between Patient and Doctor.	apptID, dateTime, status, reason
MedicalRecord	The clinical history and notes for a patient.	recordID, allergies, bloodType
Prescription	Details of medication ordered.	scriptID, dosage, duration, instructions
Room	Physical location for admitted patients.	roomNum, type (ICU/Ward), availability
Bill	Invoice generated for services.	billID, totalAmount, paymentStatus, dateIssued

Introduction to software engineering project

Deliverable three

UML Class Diagram

This diagram represents the static structure of the system, illustrating associations, cardinalities (multiplicity), and generalizations based on the Final Domain Class List above.



Introduction to software engineering project

Deliverable three

UML Class Diagram – Explanation

To justify the structural decisions made in the diagram above, the following design principles were applied:

- Key Classes: The model centers on Patient and Doctor as the primary actors. The Appointment class serves as the temporal link between them. We introduced Consultation to separate the scheduling aspect (Appointment) from the clinical aspect (Diagnosis/Notes).
- Generalization (Inheritance): We utilized an abstract Person class to hold common attributes like name, address, and phone. Both Patient and Staff inherit from Person. Furthermore, Staff is a parent class for Doctor and Nurse. This reduces redundancy and allows for shared operations like login ().
- Composition: A strong "part-whole" relationship exists between Patient and MedicalRecord. If a Patient is deleted from the system, their Medical Record implies no semantic meaning on its own; thus, we used Composition (*--).
- Aggregation: We applied aggregation (o--) between Prescription and Medicine. While a prescription lists medicines, the Medicine (inventory item) exists independently of any single prescription.

Modeling Assumptions

The following assumptions were made to define the boundaries and cardinalities of the domain model:

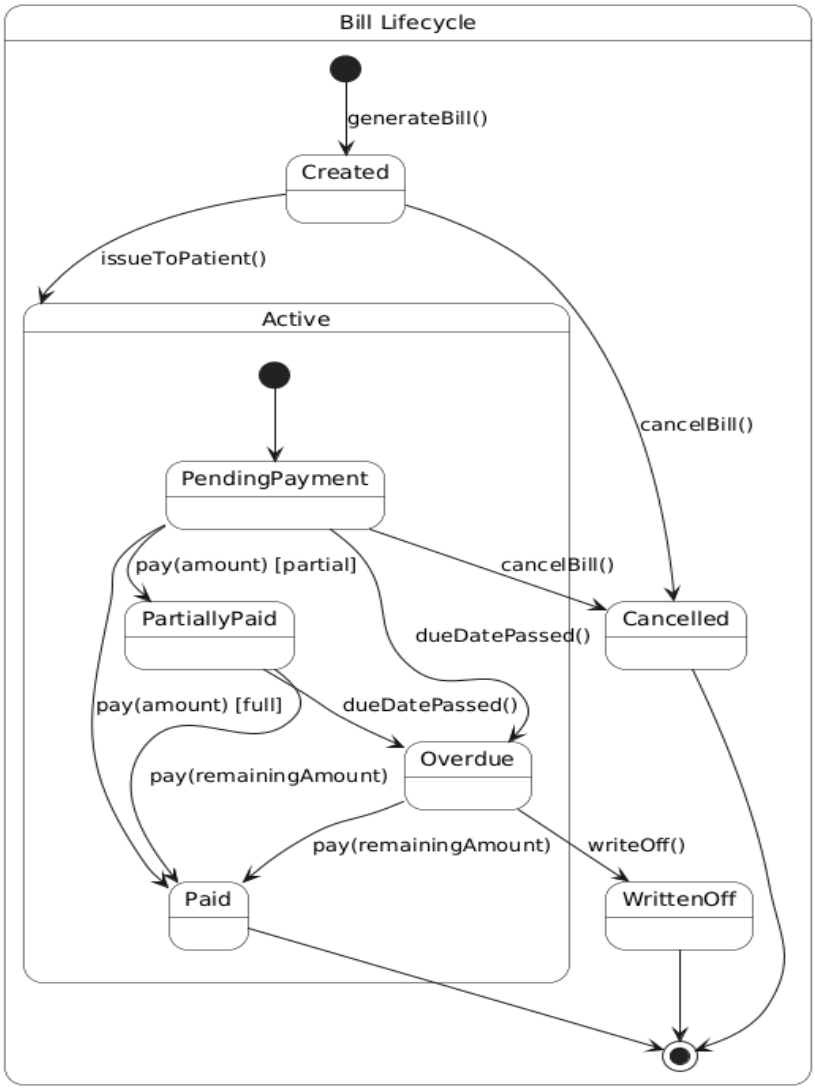
- Single Medical Record: Each patient has exactly one MedicalRecord that aggregates all their history; they do not have separate records for different visits.
- Appointment Exclusivity: An Appointment is strictly between one Doctor and one Patient at a time (no group therapy sessions).
- Billing Trigger: A Bill is generated specifically from a Consultation, not just from an appointment request.
- Prescription Items: A single Prescription can include multiple Medicine items, but must include at least one (Multiplicity 1. *).

State Machine Diagram

Selected Complex Object: Bill
Rationale: The Bill object is ideal for state modeling because it passes through clearly defined business states such as creation, payment, overdue handling, cancellation, and write-off.

Introduction to software engineering project

Deliverable three



Brief Explanation

The state machine above models the lifecycle of a Bill object. A bill is first created from a completed consultation and issued to the patient entering the Pending Payment state. From there it may become Partially Paid or fully Paid, or transition to Overdue when the due date passes without full payment. Administrative actions can cancel a bill, while long-overdue bills are eventually Written Off. This lifecycle is directly driven by the Make payment, Generate monthly bill, and maintenance use cases (e.g., cleanup of expired or uncollectible bills).

Introduction to software engineering project

Deliverable three

Use Cases, CRUD & State Machines – Explanation

To justify the analytical and modeling decisions presented below, the following methodological principles were applied:

• Methodology and Scope:

We gathered DV2 use cases and confirmed both actors and the system boundary. The Noun Technique was applied to derive an aligned class set consistent with the team model. CRUD analysis was then used to map Create/Read/Update/Delete responsibilities across use cases and classes. Gaps such as missing C/U/D operations or overloaded use cases were identified, leading to refinement proposals. Use case names and briefs were updated, and corresponding UML use case diagram adjustments were planned. In addition, a second State Machine (Appointment) was designed with composite and concurrent regions. Report #1 sections were authored for CRUD findings, refinements, and short state-machine explanations.

Modeling Steps and Assumptions

The following structured steps were used to define CRUD coverage, use case interactions, and domain participation:

• Step 1 – Domain Classes:

Patient, Doctor, Appointment, MedicalRecord, Prescription, Room, Bill (Invoice).

• Step 2 – Use Cases:

Book appointment; Register patient information; Manage doctor schedule; Confirm appointment slot; Complete patient treatments; Cancel appointment; Make payment; Authorize patient discharge; Send appointment reminders; Generate weekly report schedule; Generate monthly bill; Clean up expired slots; Handle payment failure; Update slot availability; Synchronize patient information.

• Step 3 – CRUD Matrix:

See table below .

Class \ Use Case	Book appointment	Register patient information	Manage doctor schedule	Confirm appointment slot	Complete patient treatments	Cancel appointment	Make payment	Authorize patient discharge	Send appointment reminders	Generate weekly report schedule	Generate monthly bill	Clean up expired slots	Handle payment failure	Update slot availability	Synchronize patient information
Patient	R	C,U	—	R	R	R	R	R	R	—	—	—	R	—	U

Doctor	R	—	C,U	R	R	—	—	R	R	R	R	R	—	R,U	—
Appointment	C,R	—	R,U	U	R,U	R,U	R	R	R	R	R	D	R	U	—
MedicalRecord	R	C,U	—	—	U	—	—	U	—	R	—	—	—	—	U
Prescription	—	—	—	—	C,U	—	—	—	—	—	R	—	—	—	—
Room	R	—	C,U	R	R	U	—	U	—	R	—	U	—	U	—
Bill (Invoice)	—	—	—	—	—	—	C,R, U	R	—	R	C,R, U	—	U	—	—

• Step 4 – Findings:

Delete operations were centralized in Clean up expired slots. Make payment was separated for clarity. Cancel appointment was added as a distinct interaction. MedicalRecord was identified as update-only.

• Step 5 – Refinements:

Use «extend» for Handle payment failure, Update slot availability, and Synchronize patient information. Use «include» for Make payment from Book appointment when a prepayment policy applies.

Traceability Overview

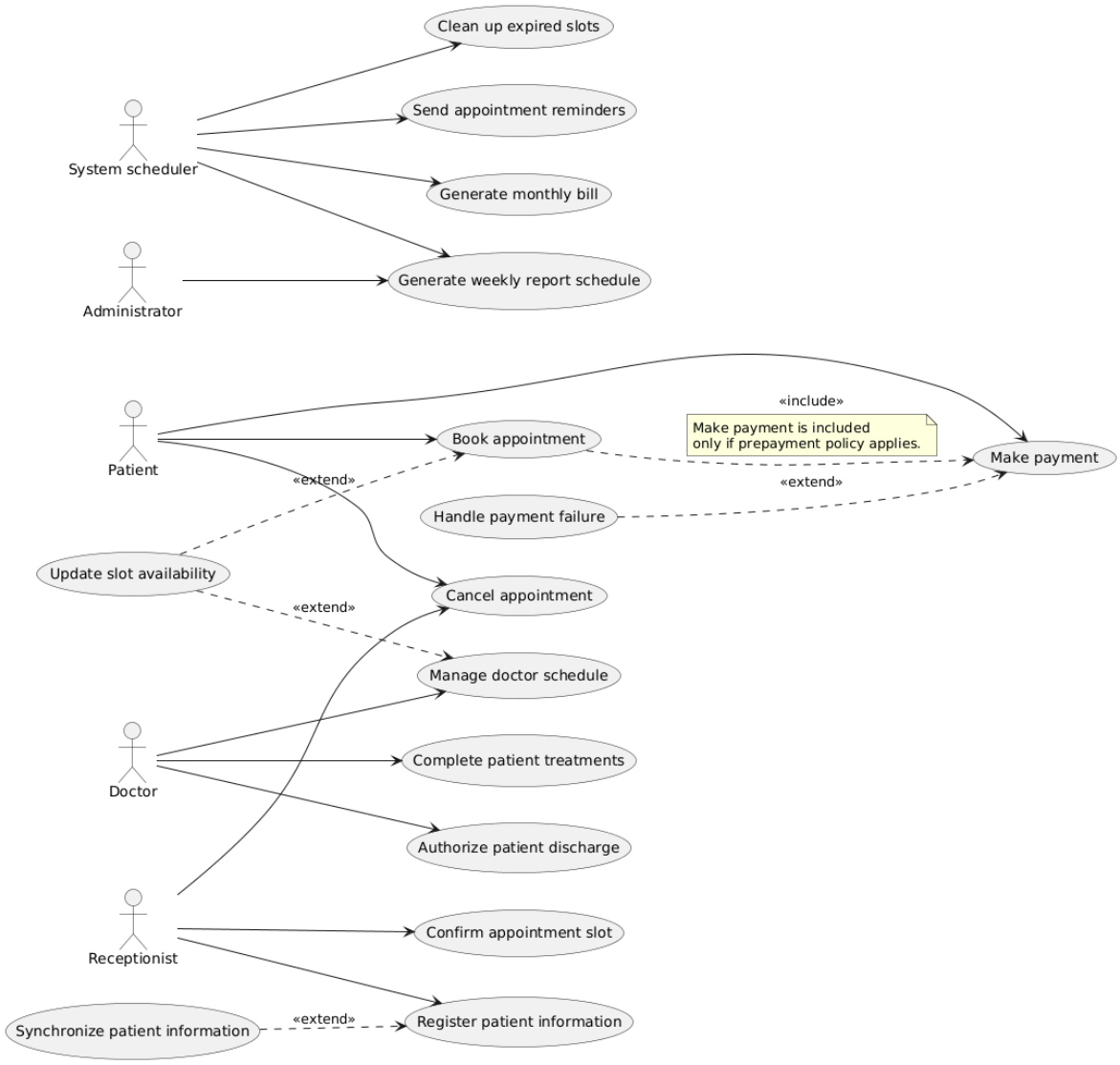
The following mapping summarizes the connections between key use cases and domain classes:

• Book appointment → Appointment, Patient, Doctor → Appointment State Machine
• Register patient information → Patient, MedicalRecord
• Complete patient treatments → MedicalRecord, Prescription
• Make payment → Bill → Bill Lifecycle State Machine
• Cancel appointment → Appointment, Room
• Authorize patient discharge → MedicalRecord, Room
• Generate monthly bill → Bill

Updated Use Case Diagram – Healthcare Management System

Introduction to software engineering project

Deliverable three



State Machine Diagrams

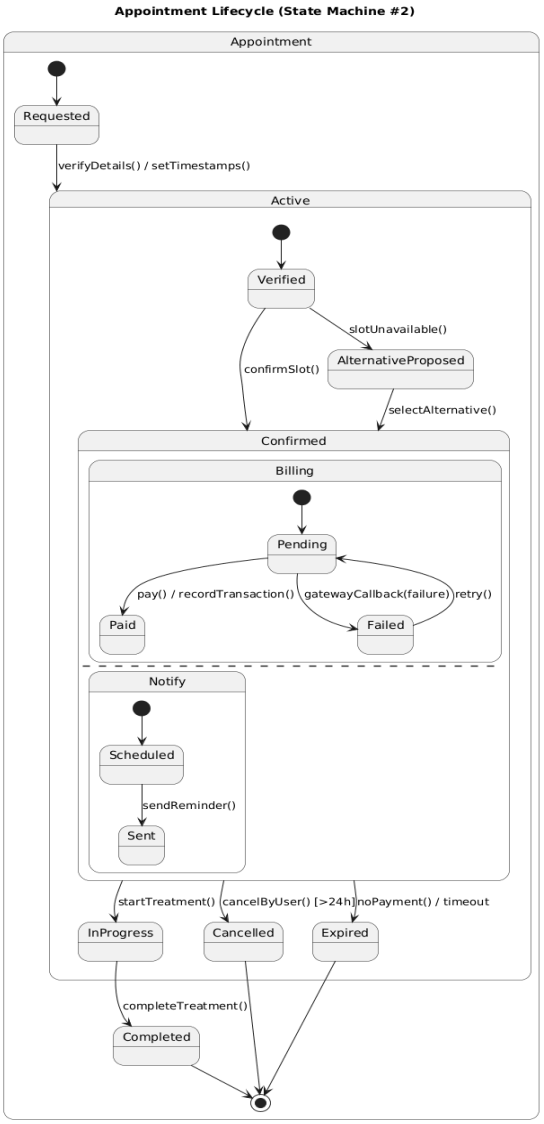
Selected Complex Objects: Bill and Appointment

• State Machine #1 – Bill Lifecycle:

This state machine models the lifecycle of a Bill, progressing through states such as Created, PendingPayment, Paid, Overdue, Cancelled, and WrittenOff. Transitions are driven by events such as bill generation after consultation, patient payments, due-date expiration, administrative cancellation, and write-off of uncollectible bills. It traces primarily to Make payment, Generate monthly bill, and billing maintenance/cleanup use cases.

• State Machine #2 – Appointment Lifecycle:

The Appointment lifecycle is modeled with a composite Active state (Verified, AlternativeProposed, Confirmed). Within Confirmed, concurrent regions capture Billing (Pending → Paid/Failed with retry) and Notifications (Scheduled → Sent). Guards such as [>24h] encode cancellation policies, no-show timeouts, and non-payment deadlines. This model maps to Book appointment, Confirm appointment slot, Complete patient treatments, Cancel appointment, Send appointment reminders, and Clean up expired slots.



State Machine #2 – Appointment Lifecycle (Explanation)

This state machine models the full lifecycle of an Appointment in the Healthcare System. An appointment is first created in the Requested state when a booking request is made. Inside the composite Active state, the system verifies the request; if the chosen slot is unavailable, the flow passes through AlternativeProposed until a new slot is selected. Once the appointment is accepted, it moves to Confirmed, which contains two concurrent regions: Billing (tracking payment from Pending through Paid or Failed with retry) and Notify (scheduling and sending reminders).

From Confirmed, the appointment can progress to InProgress and then Completed when the consultation is finished, or it can transition to Cancelled (e.g., cancelByUser()[>24h]) or Expired if no payment or confirmation occurs before a timeout. This lifecycle ties directly to the Book appointment, Confirm appointment slot, Cancel appointment, Complete patient treatments, Send appointment reminders, and billing-related use cases, ensuring that all temporal and payment-related behaviors of appointments are explicitly captured.

Use Case Explanations – Selected Complex Use Cases

To justify the selection and modeling approach for the three complex use cases below, the following criteria were applied: operational complexity, branching logic, and interaction with multiple domain objects.

- **UC-01 Book appointment**
- **UC-06 Cancel appointment**
- **UC-07 Make payment**

UC-01 Book Appointment – Detailed Textual Flow

Use Case ID: UC 01
Name: Book appointment
Primary Actor: Patient
Supporting Actors: System (Hospital System)
Goal: The patient secures an available appointment slot with a doctor.

Preconditions

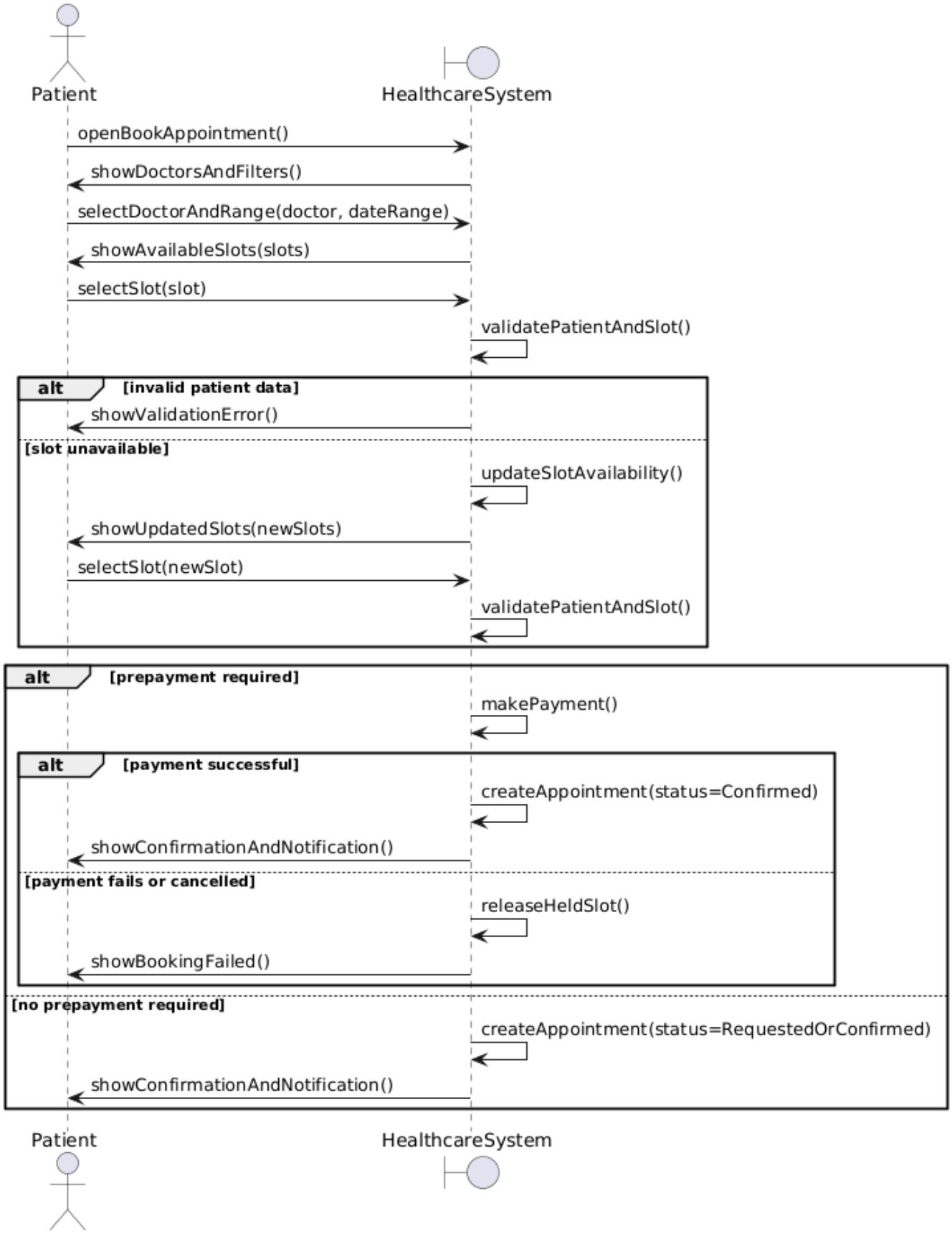
- Patient exists in the system (registered) or logs in successfully.
- Doctor schedules (available slots) are already defined.

Postconditions (Success)

- An Appointment is created with status Requested or Confirmed (policy-dependent).
- Associated Room / slot is reserved.
- Optionally, a Bill is created and payment is recorded if a prepayment policy applies.

Introduction to software engineering project

Deliverable three



UC-06 Cancel Appointment – Detailed Textual Flow

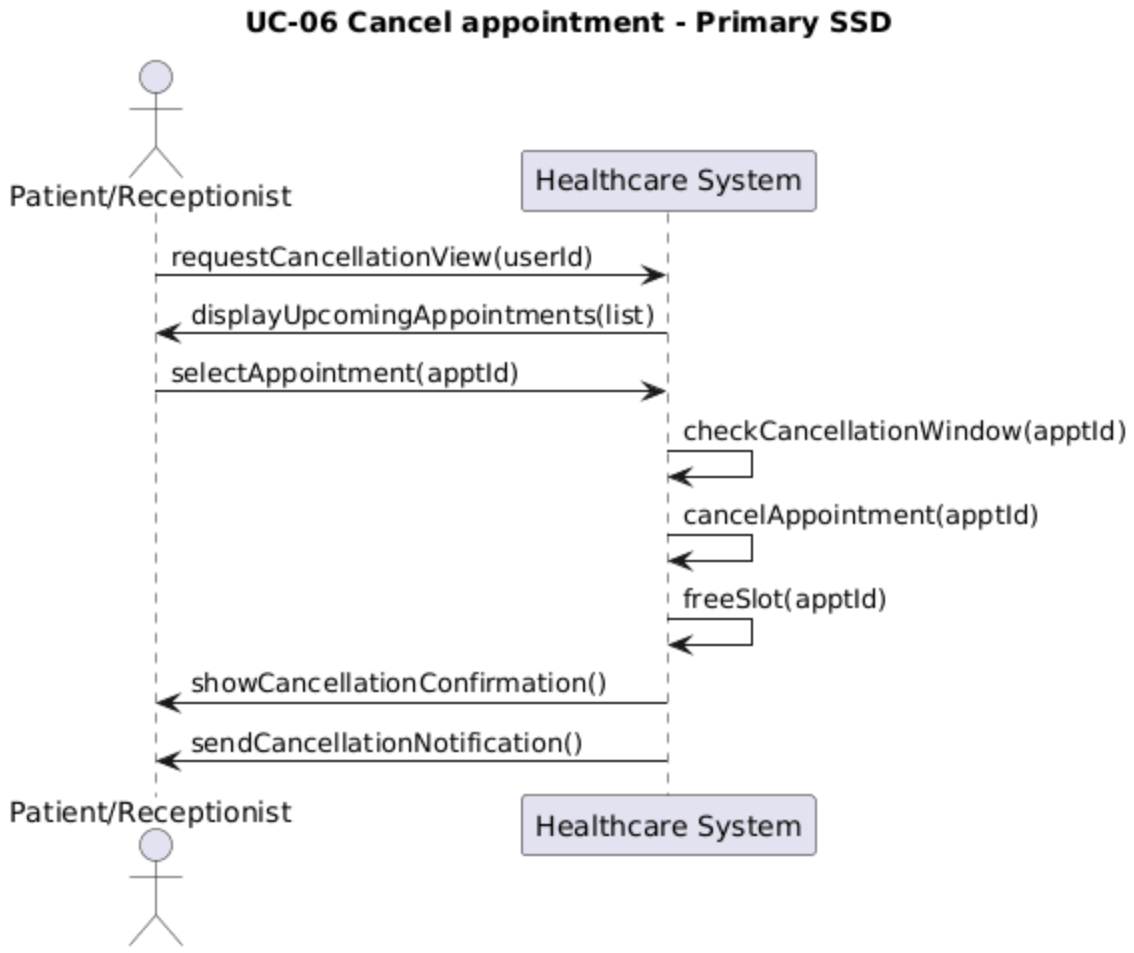
Primary Actor: Patient or Receptionist
Goal: Cancel an existing appointment while respecting the cancellation policy.

Preconditions

• Appointment exists with status **Confirmed** or **Scheduled**.
• Patient is authenticated or the receptionist is logged in.

Postconditions (Success)

• Appointment status is updated to **Cancelled**.
• Associated Room / slot is released.
• Patient and Doctor receive cancellation notifications.



Introduction to software engineering project

Deliverable three

UC-07 Make Payment – Detailed Textual Flow

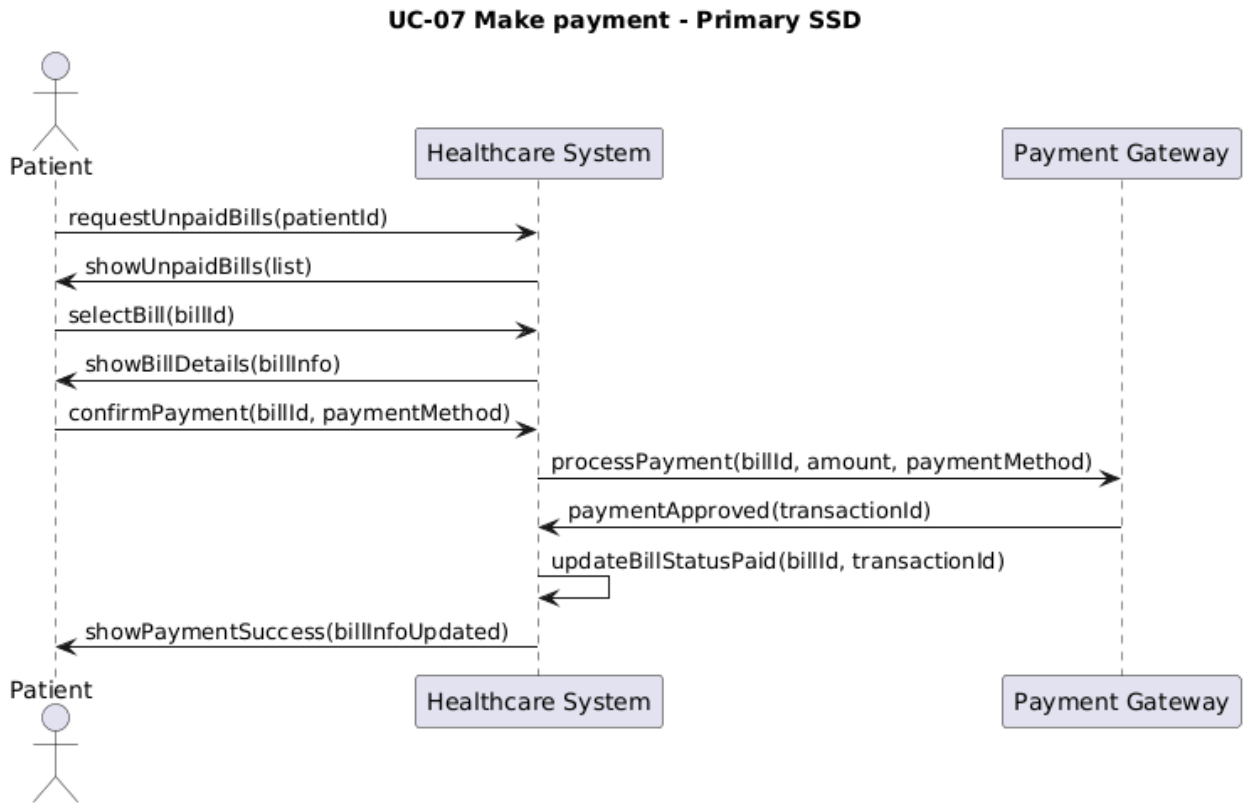
Primary Actor: Patient
Supporting Actors: Payment Gateway
Goal: Settle a bill for services rendered.

Preconditions

• A Bill exists with status **PendingPayment**.
• Patient is authenticated and can access the bill.

Postconditions (Success)

• Bill status is updated to **Paid**.
• Payment record is stored.
• Appointment / treatment is marked as **financially cleared**.



Activity Diagrams – Explanations for Report

The following activity diagrams were developed to illustrate control flow, decision logic, and system–actor interactions for each complex use case. The diagrams highlight branching conditions, exception handling, and synchronization points where system behaviors (e.g., validations, notifications, gateway calls) occur.

UC-01 Book Appointment – Activity Diagram Explanation

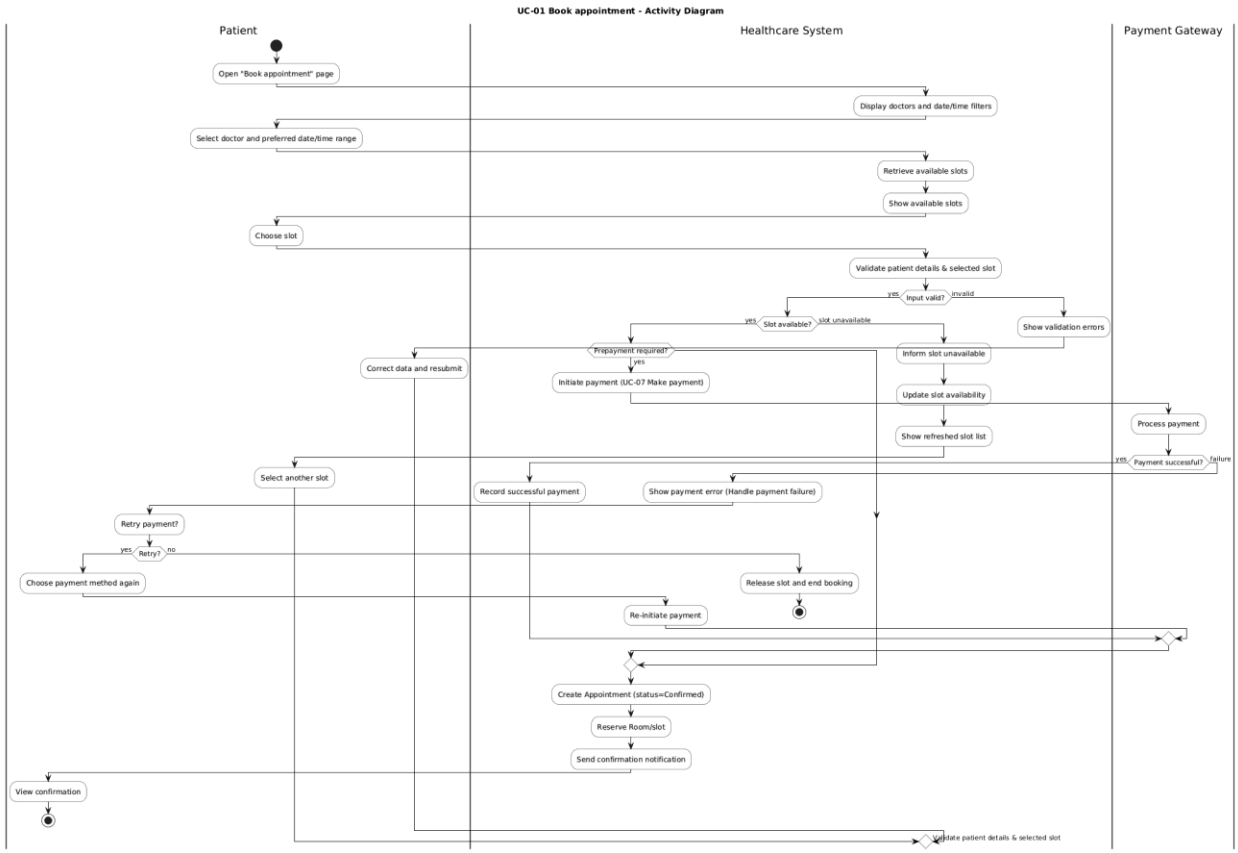
Primary Flow Activity Diagram

The activity diagram for **Book appointment** models the operational steps undertaken by the Patient and the Healthcare System to secure an appointment slot.
The flow begins when the Patient accesses the booking function. The system presents available specialties and date filters, after which the Patient selects a doctor and a preferred time range. The system retrieves matching slots and displays them.

The Patient selects a slot, triggering system validation of both the user's information and the slot availability. A decision node evaluates whether the clinic requires prepayment; if so, the **Make payment** activity is invoked. Upon successful validation (and payment if required), the system creates the Appointment, reserves the Room/slot, updates the appointment status, and proceeds to send a confirmation notification. The flow concludes with the Patient receiving confirmation details.

Alternative Flow – Slot Unavailable

The alternative activity path captures the case where the selected slot becomes unavailable during validation. After the system detects unavailability, a branch directs the flow to notify the patient and refresh slot availability by invoking **Update slot availability**. The system presents the updated list of free slots, and the Patient re-enters the selection step. The flow rejoins the main path once a valid slot is selected, leading to appointment creation, reservation, and confirmation.



UC-06 Cancel Appointment – Activity Diagram Explanation

Primary Flow Activity Diagram

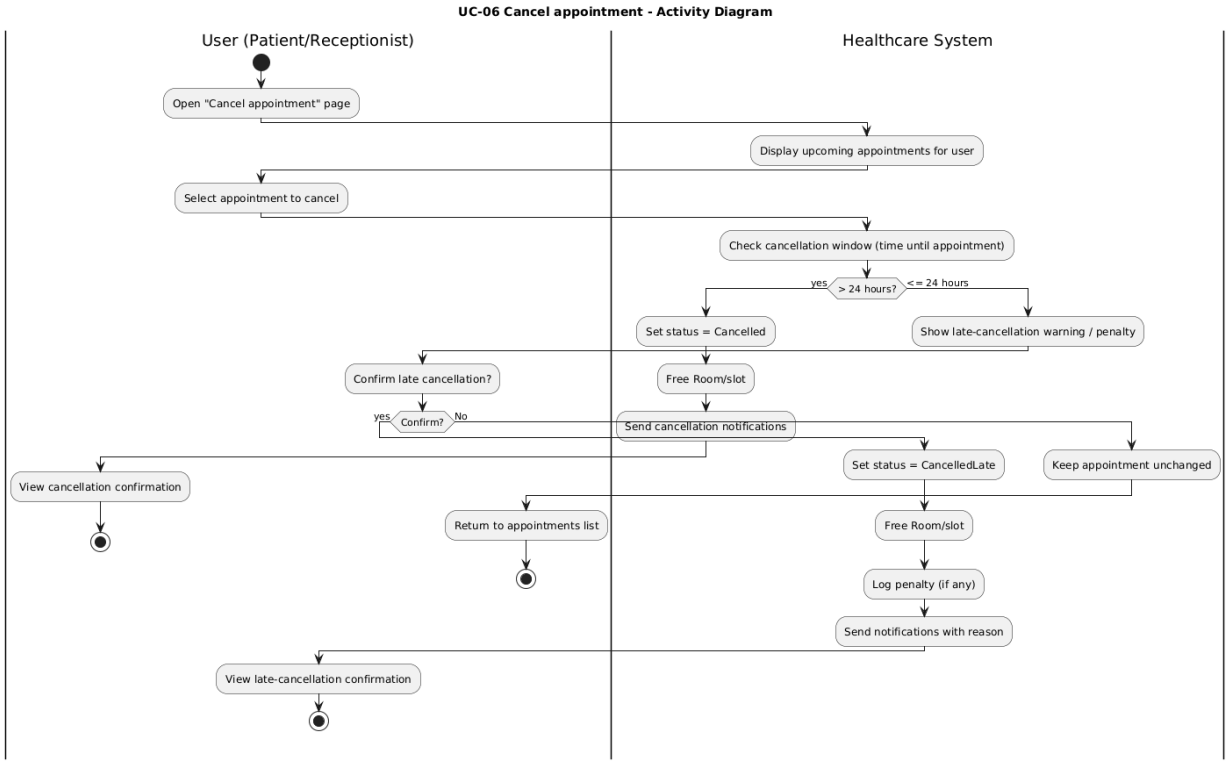
The activity diagram for **Cancel appointment** highlights the decision-based cancellation process involving either a Patient or a Receptionist. The flow begins when the actor opens the cancellation function. The system lists upcoming appointments, after which the actor chooses one to cancel.

A decision node checks the cancellation policy window (e.g., more than 24 hours before the appointment). If the cancellation is permitted, the system updates the Appointment status to **Cancelled**, frees the associated Room/slot, triggers cancellation notifications to Patient and Doctor, and displays a confirmation to the user.

Alternative Flow – Late Cancellation

The alternative activity path addresses policy violations when the cancellation request occurs too close to the appointment time. After evaluating the cancellation window, the system displays a penalty or warning prompt. A branch allows the actor to either confirm the late cancellation or

abort.
If confirmed, the system updates the appointment with a late-cancellation status, frees the slot, logs penalty details, and sends notifications including the reason.
If the actor chooses not to proceed, the system leaves the appointment unchanged and the activity flow terminates without modifications.



UC-07 Make Payment – Activity Diagram Explanation

Primary Flow Activity Diagram

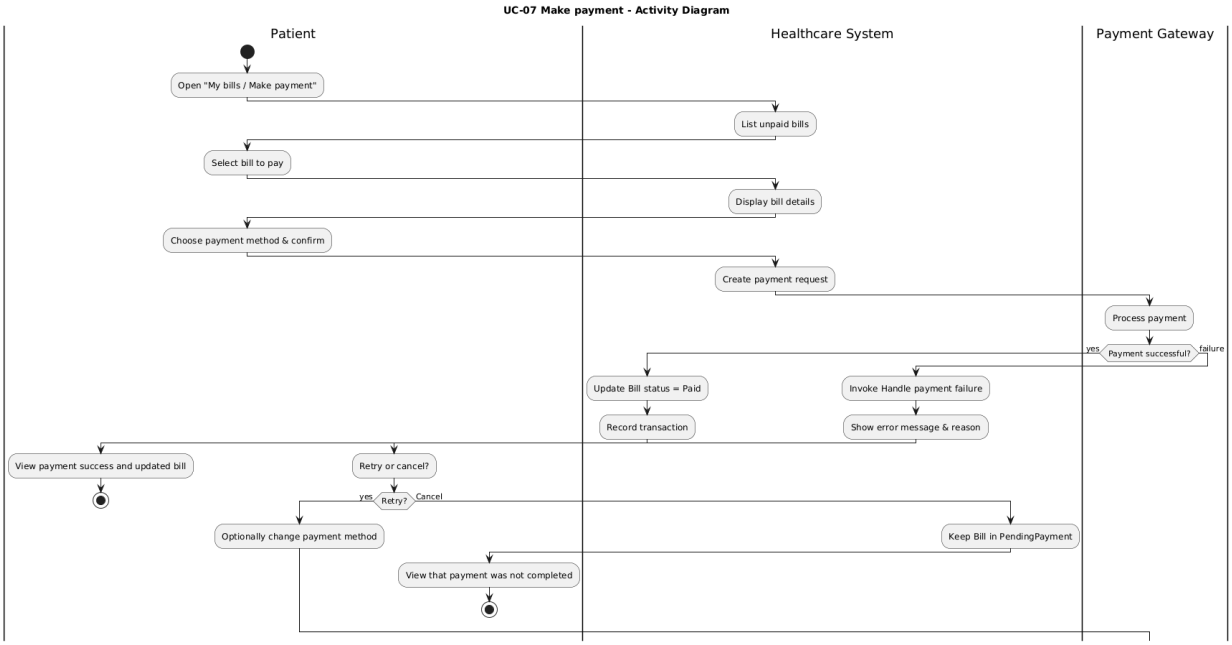
The activity diagram for **Make payment** models interaction among the Patient, the Healthcare System, and the external Payment Gateway. The flow begins when the Patient accesses their unpaid bills and selects one to settle. The system displays the bill details, and the Patient chooses a payment method and confirms.

A system action sends a payment request to the Payment Gateway. Following the external authorization, a decision node evaluates whether the result is approved. If approved, the system updates the Bill status to **Paid**, records the transaction details, and presents a success message with the updated bill status. The flow ends with the Patient viewing confirmation of successful payment.

Alternative Flow – Payment Failure

The alternative branch corresponds to the **Handle payment failure** extension. If the Payment Gateway returns a failure response, the system displays an error message and logs the failure. A user decision node offers two paths: **Retry** or **Cancel**.

If the Patient retries, the flow returns to the payment request step (potentially after choosing a different payment method). Upon eventual success, the flow rejoins the primary scenario.
If the Patient cancels, the system retains the Bill in **PendingPayment**, increments any failure counters as required, and ends the flow with the bill remaining unpaid.



Inputs for SSD Design

The remaining System Sequence Diagrams were derived from the finalized Deliverable 2 use case list and briefs, using the agreed domain model (Patient, Doctor, Appointment, MedicalRecord, Prescription, Room, Bill). We reused the same naming conventions as the other SSDs: actors such as Patient, Receptionist, Doctor, System scheduler, and Administrator; a single HealthcareSystem lifeline as the system boundary; and high-level message names (for example, requestAppointment(), showUnpaidBills(), cancelAppointment()).

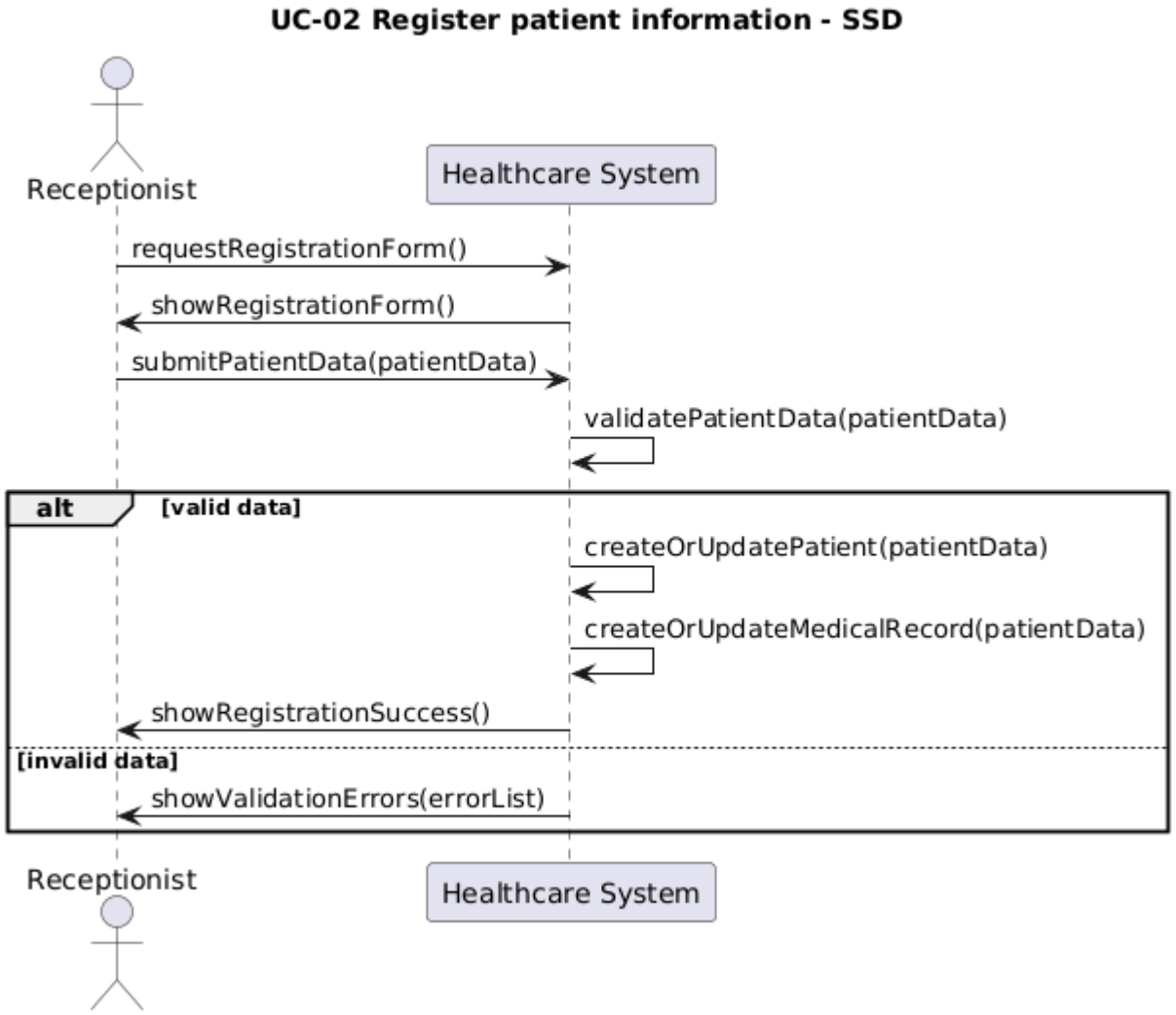
System Sequence Diagrams – Design Approach

For the remaining use cases (UC-02 to UC-12 and UC-T1–T3), System Sequence Diagrams (SSDs) were created to capture the external interactions between actors and the Healthcare System at the system boundary. Each SSD uses a single HealthcareSystem lifeline to represent the system as a black box, together with the relevant external actors such as Patient, Receptionist, Doctor, System scheduler, and Administrator. Additional external services (for example, NotificationService or ReportingService) are modeled as separate lifelines only when their behaviour is visible at the system level.

The primary scenario for each use case was taken from the refined Deliverable 2 descriptions. Individual steps in the main flow were translated into high-level messages rather than low-level UI events. Actor-initiated actions are modeled as requests to HealthcareSystem (for example, registerPatient(data), manageSchedule(), confirmSlot(), authorizeDischarge()), while system responses are modeled as messages back to the actor (for example, showForm(), displaySchedule(), showConfirmation()). Where the system invokes background or external behaviour (such as sending reminders or generating reports), additional messages are sent from HealthcareSystem to the corresponding service lifeline.

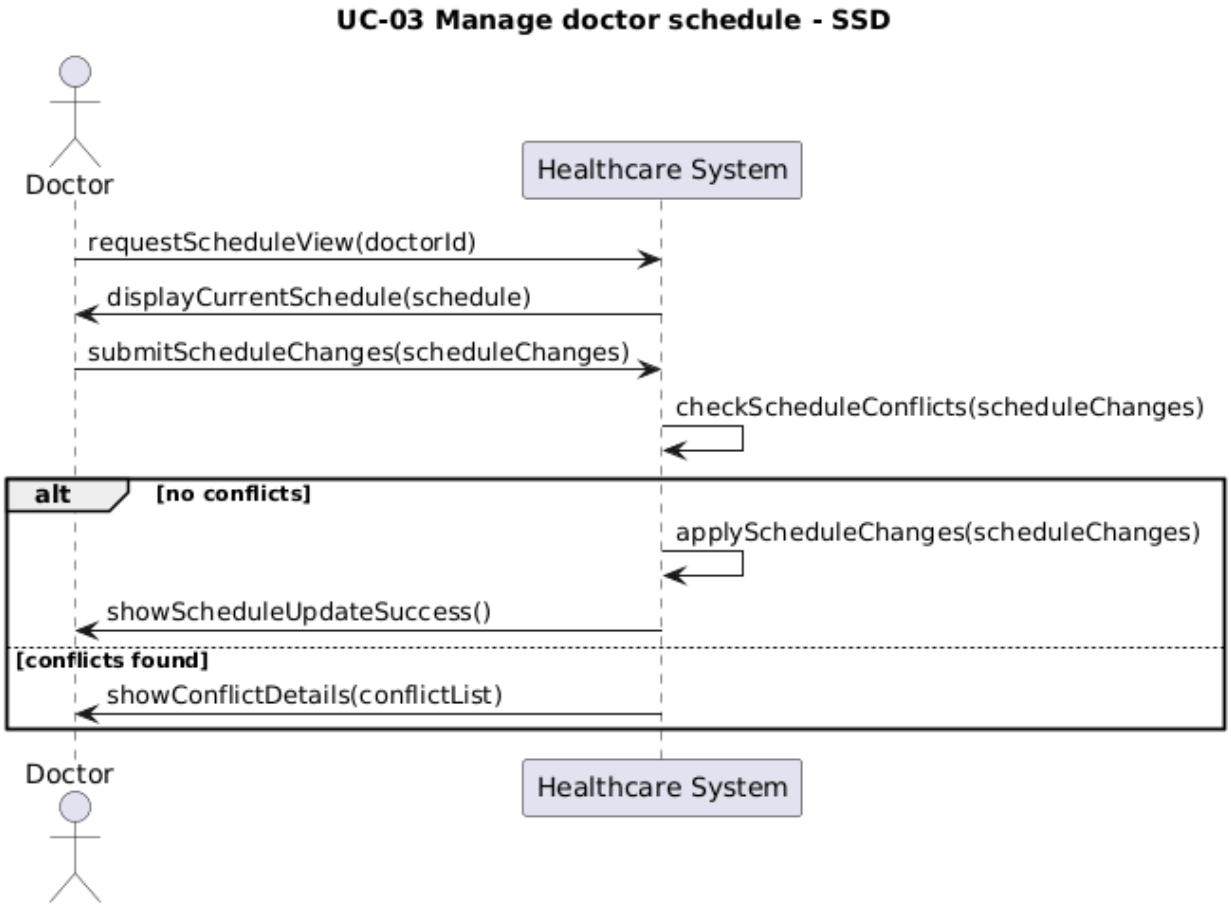
To ensure consistency across the whole model, the SSDs follow the same naming and style conventions as those prepared for the three complex use cases (UC-01 Book appointment, UC-06 Cancel appointment, UC-07 Make payment). The system lifeline is always named HealthcareSystem, message names use clear verb–object patterns (requestX(), showY(), updateZ()), and no low-level GUI clicks are shown. Each use case has at least one primary SSD covering the everything-goes-right scenario; important alternative or exception paths (such as invalid input, conflicts in schedules, or failures in background jobs) are represented using UML alt fragments within the same diagram. Diagrams are exported with consistent naming (for example, SSD_UC02_RegisterPatient_Primary, SSD_UC03_ManageSchedule_Primary) and are referenced throughout the report using these identifiers.

UC-02 Register patient information – SSD



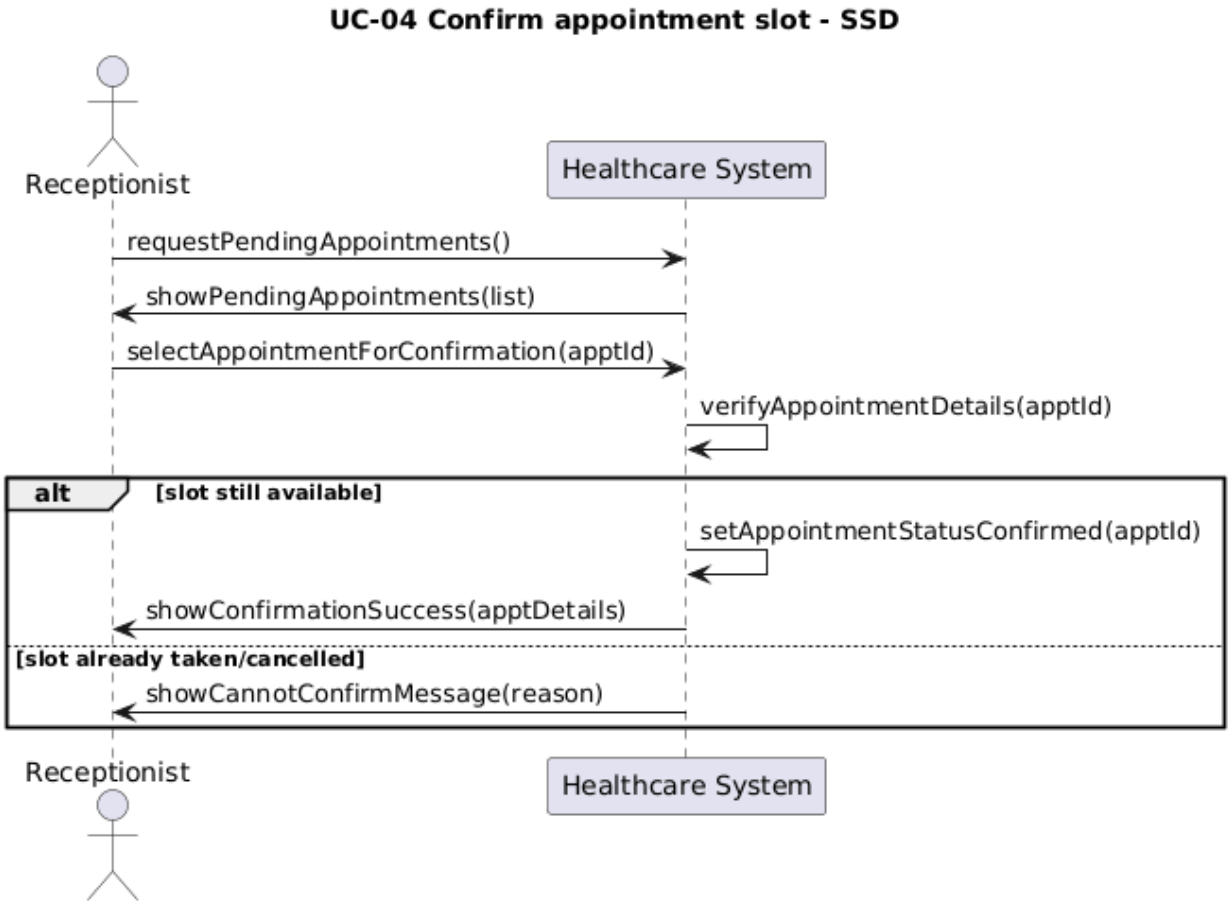
The SSD for Register patient information shows the interaction between the Receptionist and the HealthcareSystem. The receptionist requests the registration form, enters patient details, and submits them. The system validates the data; on success, it creates or updates both the Patient and the associated MedicalRecord, then returns a success confirmation. If validation fails, the alt fragment captures the alternative path where the system returns detailed validation errors so the receptionist can correct and resubmit.

UC-03 Manage doctor schedule – SSD



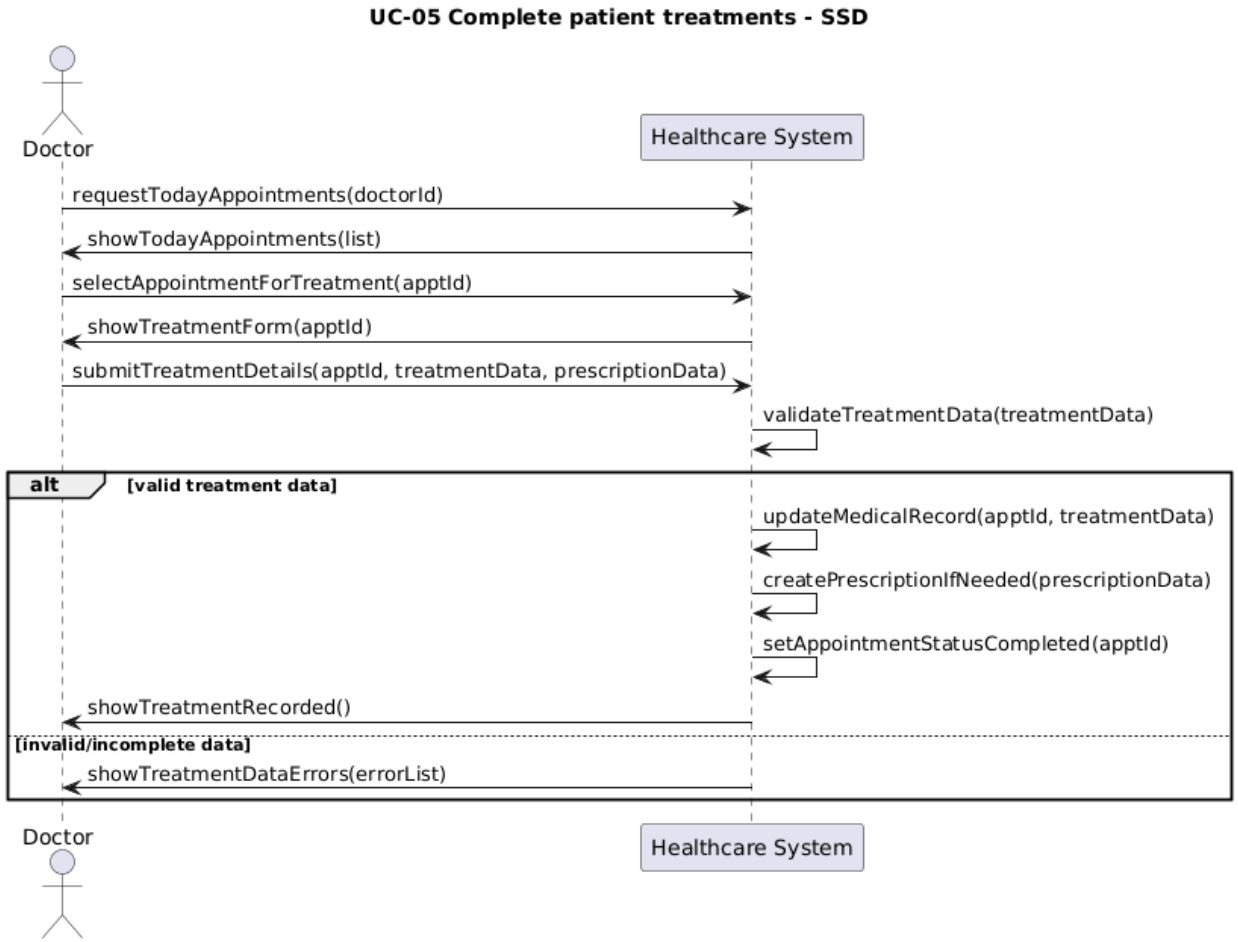
The SSD for Manage doctor schedule models how a Doctor views and updates their schedule. The doctor requests the current schedule, edits it, and submits the proposed changes. The HealthcareSystem checks for conflicts such as overlapping slots or invalid times. If no conflicts are found, the changes are applied and a success message is returned; otherwise, the alt fragment shows the system sending conflict details so the doctor can resolve them

UC-04 Confirm appointment slot – SSD



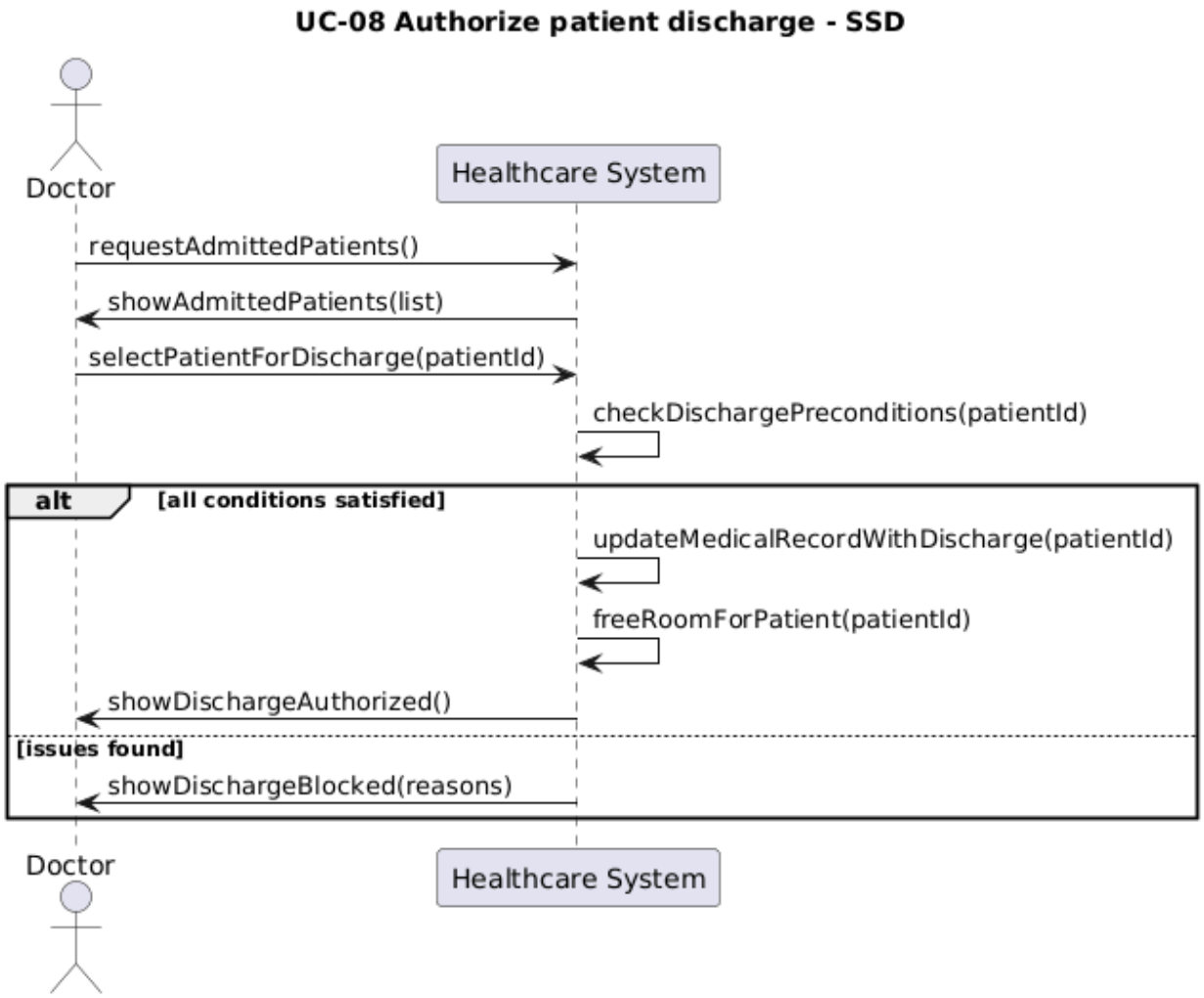
The SSD for Confirm appointment slot captures how the Receptionist confirms pending appointments. The receptionist requests the list of pending appointments, selects one, and the system verifies its details and current slot status. If the slot is still valid, the system marks the appointment as Confirmed and returns the updated details. The alt fragment covers the case where the slot has already been taken or cancelled, in which case the system cannot confirm and returns an explanatory message.

UC-05 Complete patient treatments – SSD



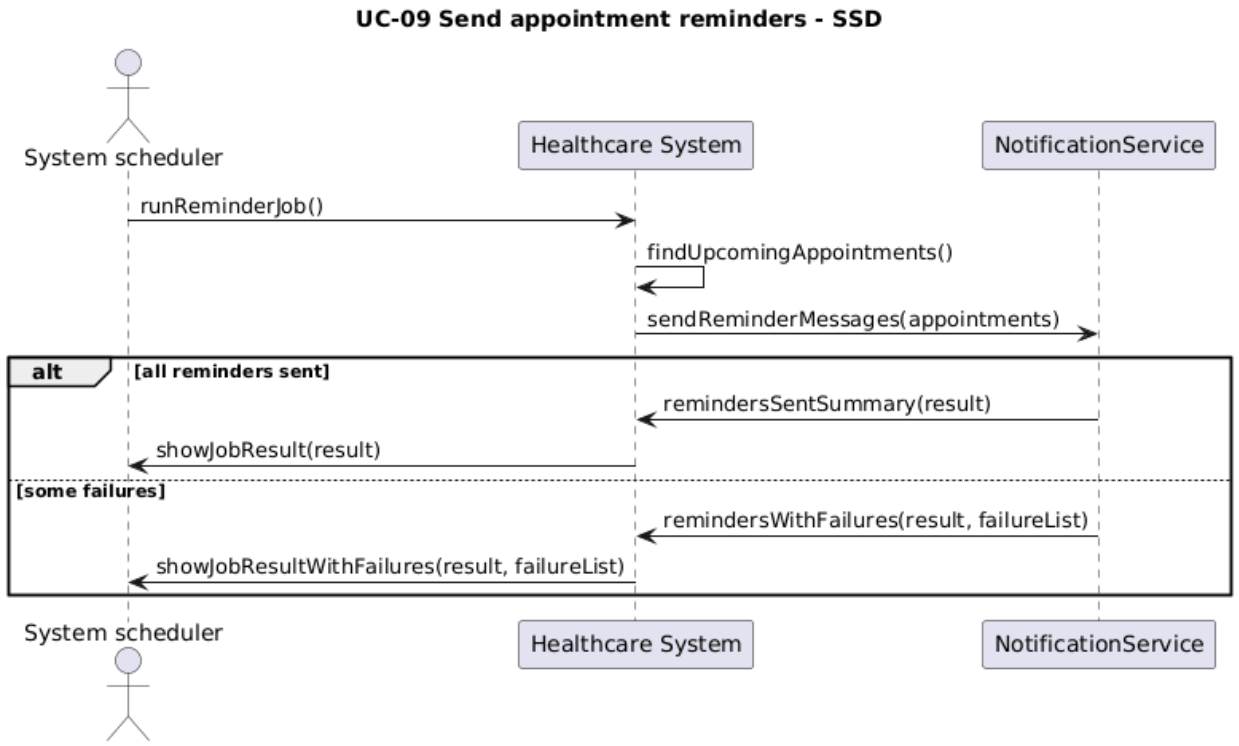
The SSD for Complete patient treatments describes how a Doctor records the outcome of a consultation. The doctor selects an appointment from today's list, fills in treatment details, and optionally adds prescription information. The HealthcareSystem validates the data and, if valid, updates the MedicalRecord, creates a Prescription when needed, and marks the appointment as Completed. The alt path shows the system returning errors when treatment data is incomplete or invalid.

UC-08 Authorize patient discharge – SSD



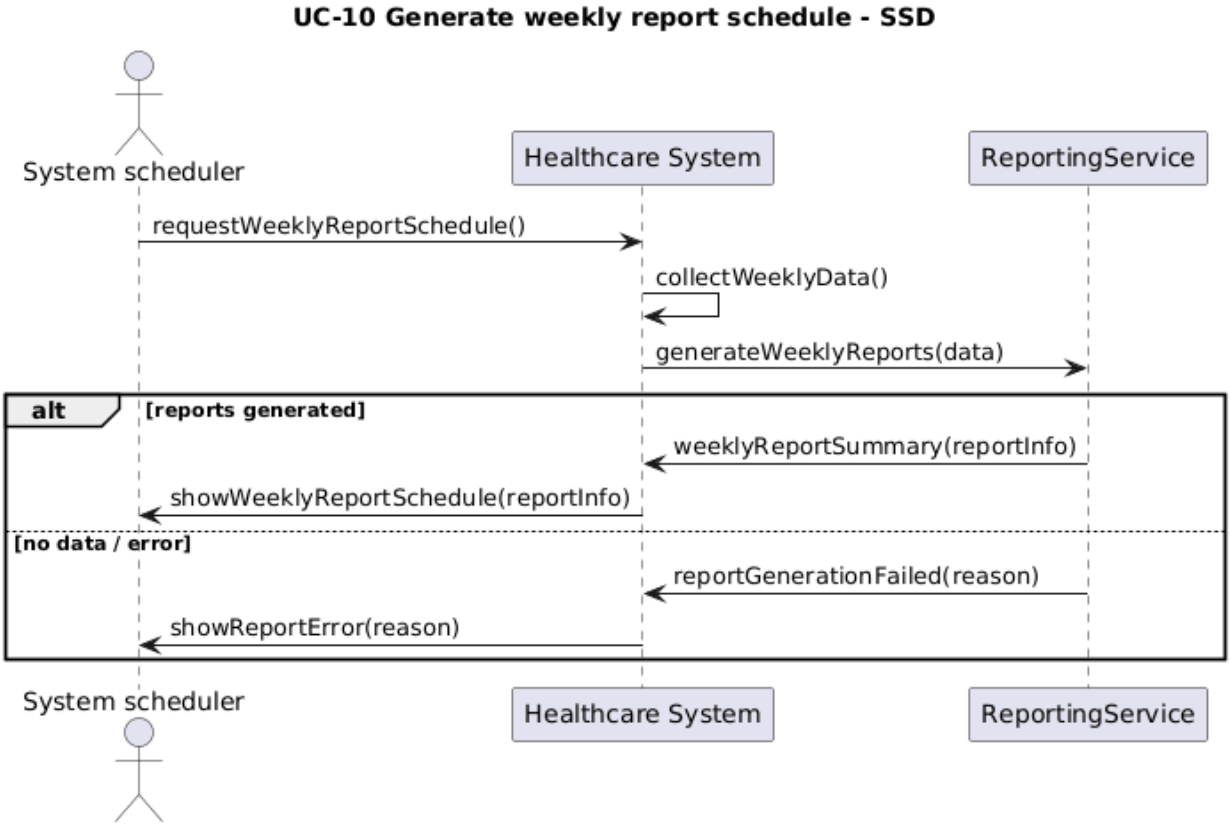
The SSD for Authorize patient discharge shows how a Doctor discharges an admitted patient. The doctor requests the list of admitted patients, selects one, and the system evaluates discharge preconditions such as completed treatments or outstanding issues. If all conditions are satisfied, the system updates the MedicalRecord, frees the assigned Room, and confirms that discharge is authorized. The alt fragment covers the case where unresolved treatments or other issues prevent discharge, and the doctor is informed of the blocking reasons.

UC-09 Send appointment reminders – SSD



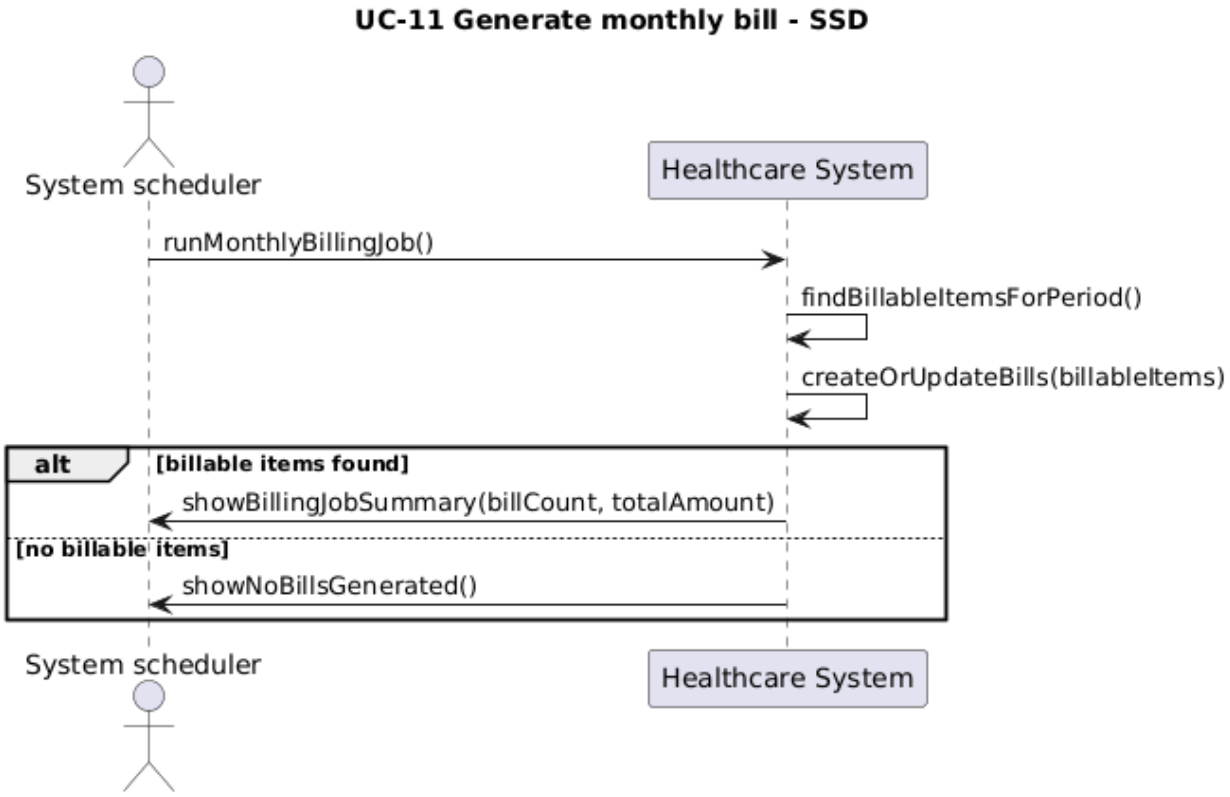
The SSD for Send appointment reminders represents the batch reminder job initiated by the System scheduler. The scheduler triggers the reminder job, the HealthcareSystem retrieves upcoming appointments, and forwards them to an external NotificationService for delivery. The main path returns a summary of successfully sent reminders, while the alt path captures partial failures, where the system reports which reminders could not be delivered so they can be investigated.

UC-10 Generate weekly report schedule – SSD



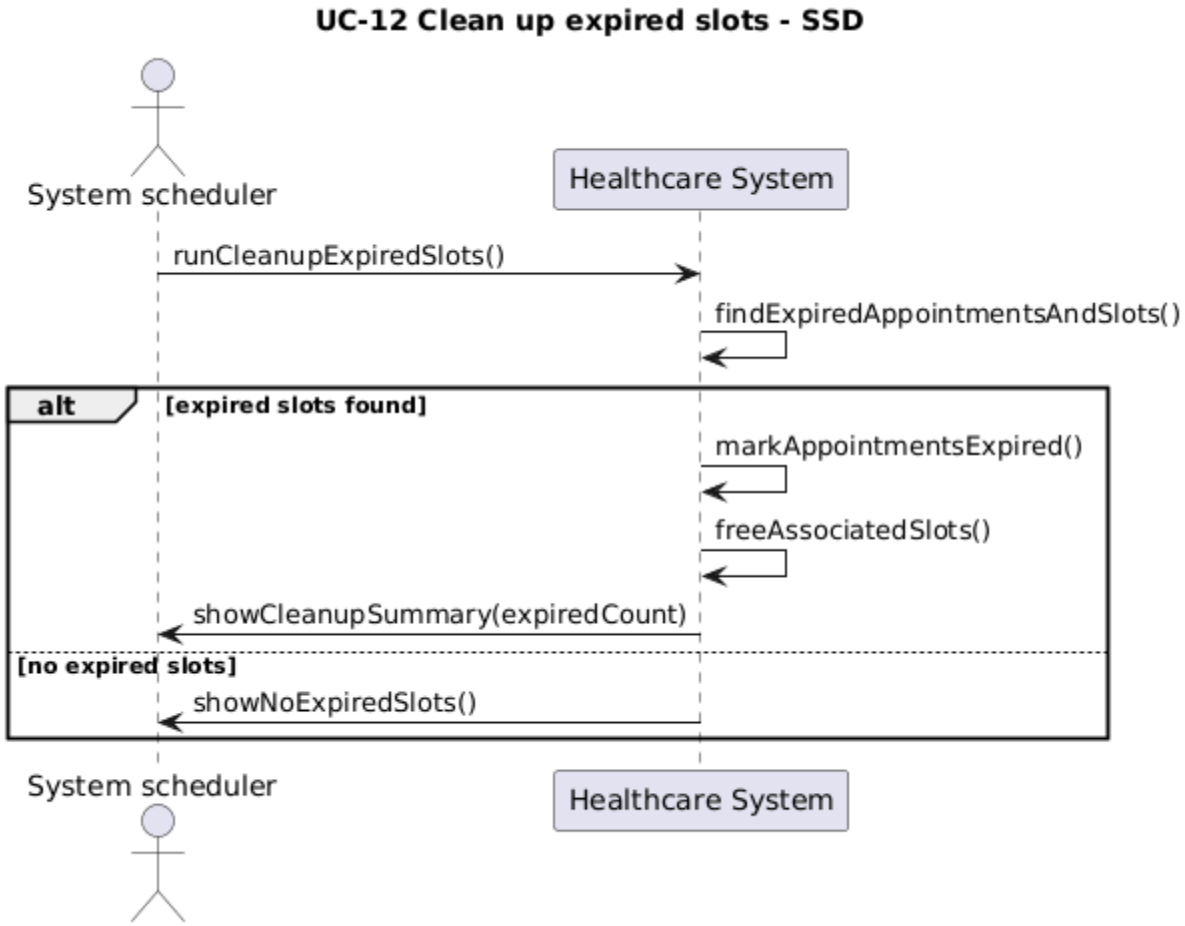
The SSD for Generate weekly report schedule shows how the System scheduler requests weekly operational reports. The HealthcareSystem collects the necessary data and delegates report generation to a ReportingService. In the main flow, the service returns a summary of generated reports, which is shown back to the scheduler. The alt fragment covers the case where report generation fails or there is insufficient data, in which case the scheduler is informed of the error.

UC-11 Generate monthly bill – SSD



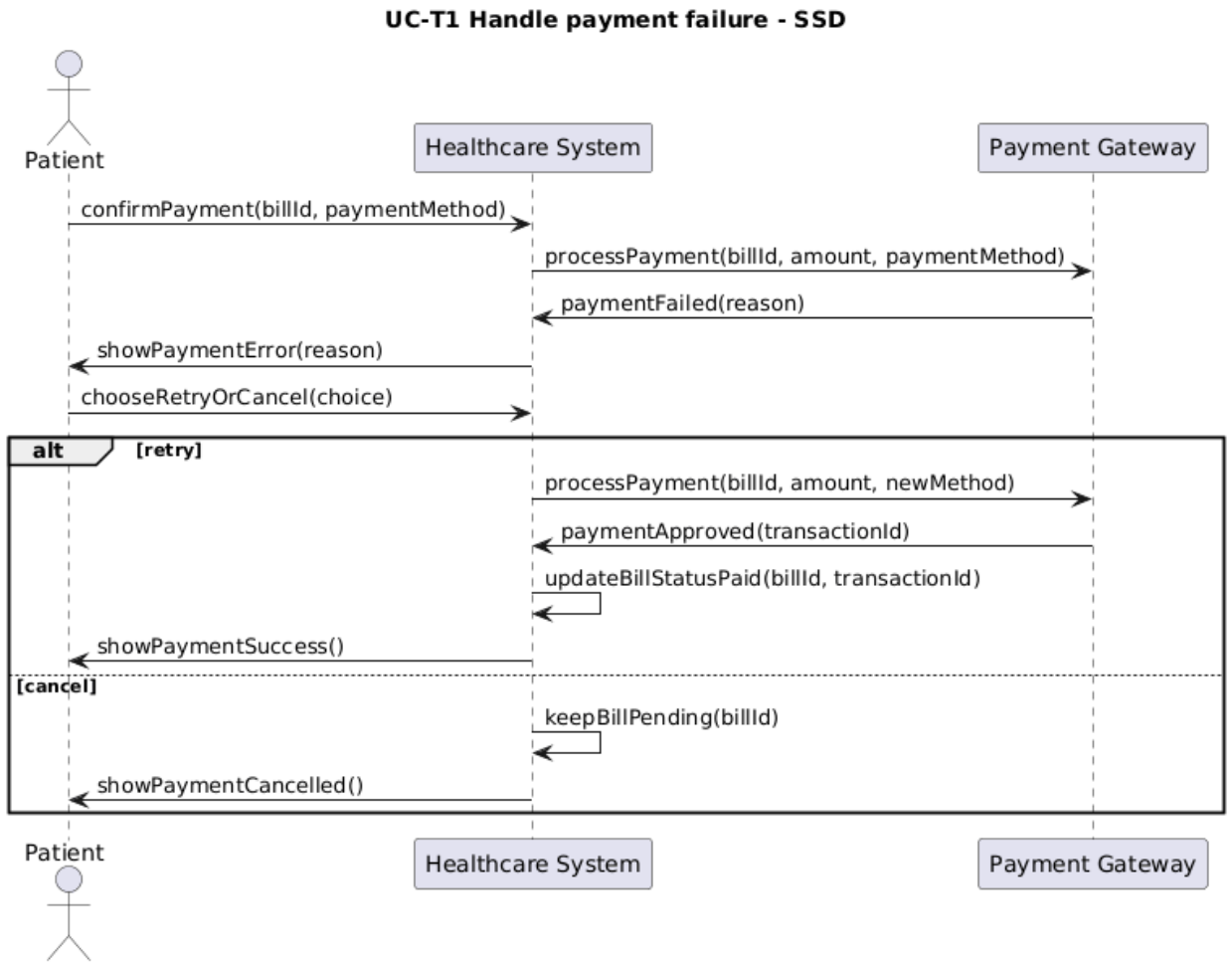
The SSD for Generate monthly bill models the automated billing job started by the System scheduler. The HealthcareSystem identifies billable items (such as completed treatments) for the target period and creates or updates Bill records accordingly. The main path reports how many bills were generated and the total amount. The alt branch shows the case where no billable items are found, and the scheduler is informed that no bills were created

UC-12 Clean up expired slots – SSD



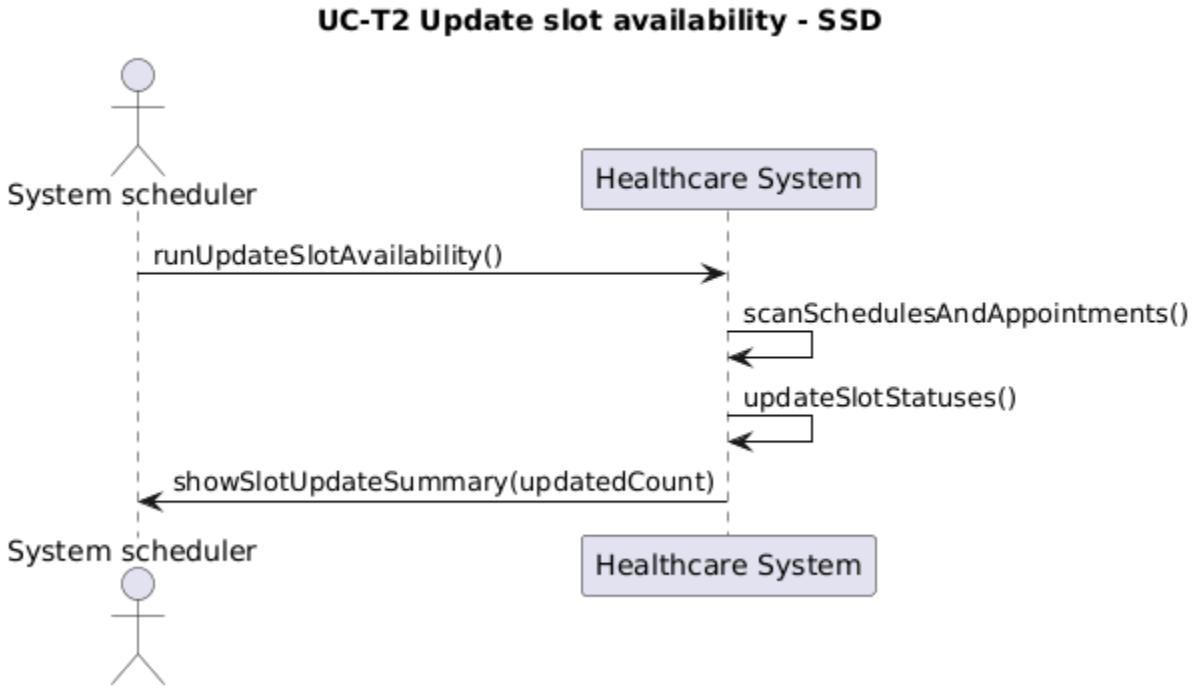
The SSD for Clean up expired slots describes a maintenance job triggered by the System scheduler. The HealthcareSystem locates appointments and slots that have passed their expiry conditions. If expired slots are found, the system marks the related appointments as Expired, frees the associated resources, and returns a cleanup summary. The alt path shows the simpler case where no expired slots exist and the scheduler is informed accordingly.

UC-T1 Handle payment failure – SSD



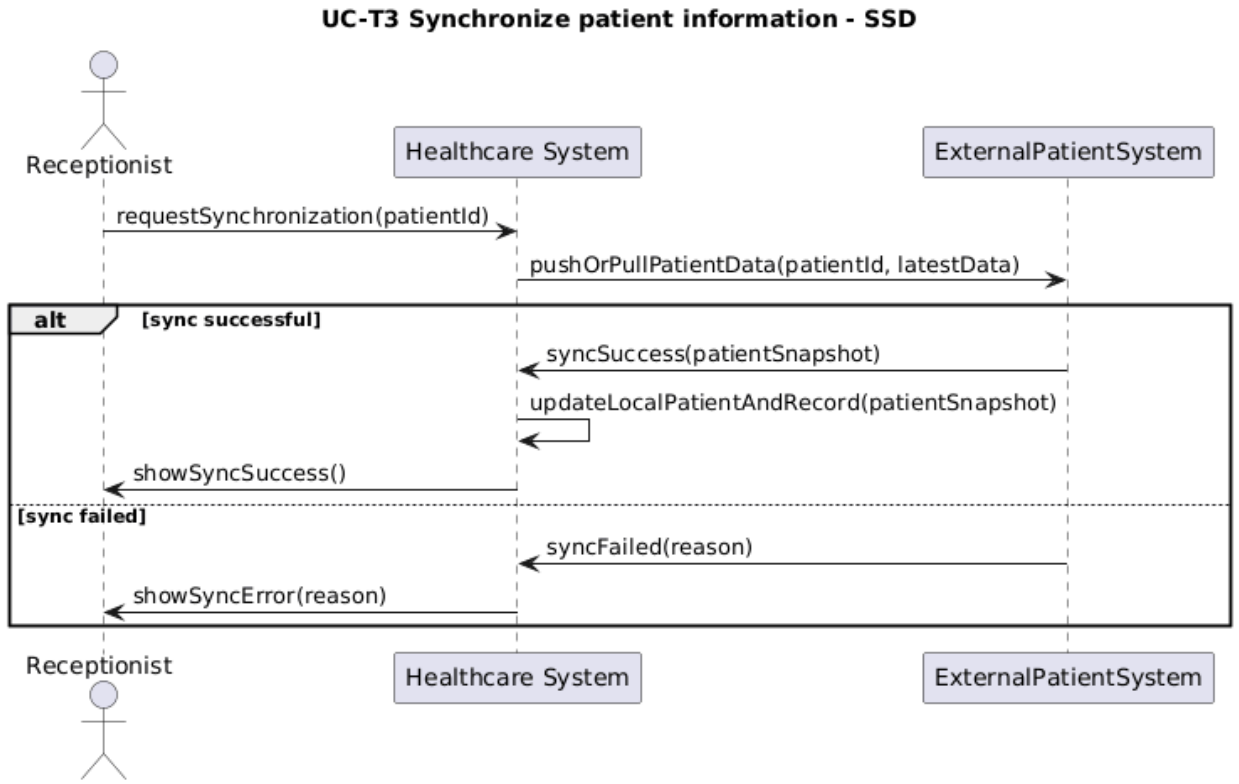
The SSD for Handle payment failure focuses on the error path of the payment process. After the Patient confirms payment, the HealthcareSystem calls the external Payment Gateway, which returns a failure. The system displays the error and asks the patient to retry or cancel. The alt fragment captures the two outcomes: on retry, a new payment attempt may succeed and the bill is marked Paid; on cancel, the bill remains in a pending state and the patient is informed that payment was not completed.

UC-T2 Update slot availability – SSD



The SSD for Update slot availability represents a maintenance use case that keeps appointment slots synchronized with the latest schedule information. The System scheduler triggers the update, and the HealthcareSystem scans existing schedules and appointments, adjusts slot statuses (for example, marking slots as free, reserved, or blocked), and returns a summary of how many slots were updated. This behavior is invoked indirectly by Book appointment and Manage doctor schedule as an extension use case.

UC-T3 Synchronize patient information – SSD



The SSD for Synchronize patient information covers the extension use case that keeps local patient data consistent with an external system. The Receptionist initiates synchronization for a given patient, the HealthcareSystem exchanges data with an ExternalPatientSystem, and then either updates the local Patient and MedicalRecord or reports an error. The alt fragment distinguishes successful synchronization from failures such as connectivity or validation problems.

Alternative and Exception Scenarios in SSDs

In addition to the primary “everything-goes-right” flows, the System Sequence Diagrams for the remaining use cases also capture important alternative and exception behavior using UML alt fragments. For example, the SSD for UC-02 Register patient information distinguishes between valid and invalid patient data; UC-03 Manage doctor schedule models the case where proposed schedule changes introduce conflicts; UC-04 Confirm appointment slot shows what happens

when the selected slot has already been taken; and UC-05 Complete patient treatments includes the branch where treatment data is rejected.

Batch and maintenance use cases follow the same pattern. The SSDs for UC-09 Send appointment reminders, UC-10 Generate weekly report schedule, UC-11 Generate monthly bill, and UC-12 Clean up expired slots each include an alternative path when no data is found or when partial failures occur. The extension use cases UC-T1 Handle payment failure and UC-T3 Synchronize patient information explicitly model retry and failure branches, while UC-T2 Update slot availability is a simple maintenance operation that only requires a primary scenario. These alternative paths ensure that the SSDs are aligned with the refined use case descriptions and fully describe both normal and exceptional interactions with the HealthcareSystem.

Workload

Ahmed **– Domain Classes & Class Diagram Lead**

- **Domain classes (Noun Technique)**
    - Extract all nouns from case study + previous deliverables.
    - Build all required tables:
        - Raw noun list.
        - Classification (class/attribute/actor/irrelevant/synonym).
        - Final domain class table with attributes and (if needed) operations.
    - Write the **Noun Technique section** of Report #1 (explain steps + rationale).
- **UML class diagram**
    - Design the full class diagram:
        - Classes, attributes, key operations.
        - All associations + cardinalities.
        - Generalization, aggregation, composition.
    - Keep diagram consistent with the Noun Technique tables.
- **State machine #1**
    - Co-select 2 complex data objects with Member 2.

- Create **state machine diagram** for object #1.
- Explain it briefly in Report #1.

Andrew - Use Cases, CRUD & State Machines Lead

- **CRUD technique**
  - Gather all use cases from Deliverable 2.
  - Build the **CRUD matrix** (rows: classes, columns: use cases).
  - Analyze:
    - Missing create/update/delete operations.
    - Missing or overloaded use cases.
  - Propose refinements/new use cases.
- **Update use case model**
  - Update **use case descriptions** (brief text).
  - Update the **UML use case diagram** based on CRUD results.
- **State machine #2**
  - Choose complex data object #2 (with Member 1).
  - Create the second **state machine diagram** with detailed states and transitions.
- **Report #1 sections**
  - Write the **CRUD + use case refinement** part of Report #1.
  - Add short explanation for both state machines.

Adham – Activity Diagrams & Part of SSDs

- **Select complex use cases**
  - With Member 2, pick **3 complex use cases**.
- **Activity diagrams (3)**
  For those 3 use cases:

    - Identify main + alternative flows.
    - Create **activity diagrams with swimlanes** (actors vs system, etc.).
    - Export diagrams.
- **SSDs for the 3 complex use cases**
    - For each of the 3:
        - Model **primary scenario SSD**.
        - Model **alternative/exception SSDs** (separate diagrams if clearer).
        - Model 4 more SSDs
- **Report #1 sections**
    - Write the **activity diagrams + SSDs (for these 3 use cases)** part of Report #1.
- Project coordination
    - Set up and maintain Jira:
        - Create tasks matching these responsibilities.
        - Track progress.
    - Set up and maintain GitHub repo:
        - Folder structure for UML models, diagrams, Report #1, SRS.
        - Make sure everyone's diagrams and docs are pushed.
    - Final packaging & submission:
        - Collect final PDFs (Report #1 + SRS).
        - Zip UML project and any required files.
    - Check guidelines and upload them to Canvas.

Yassin – SRS, Remaining SSDs & Project Management

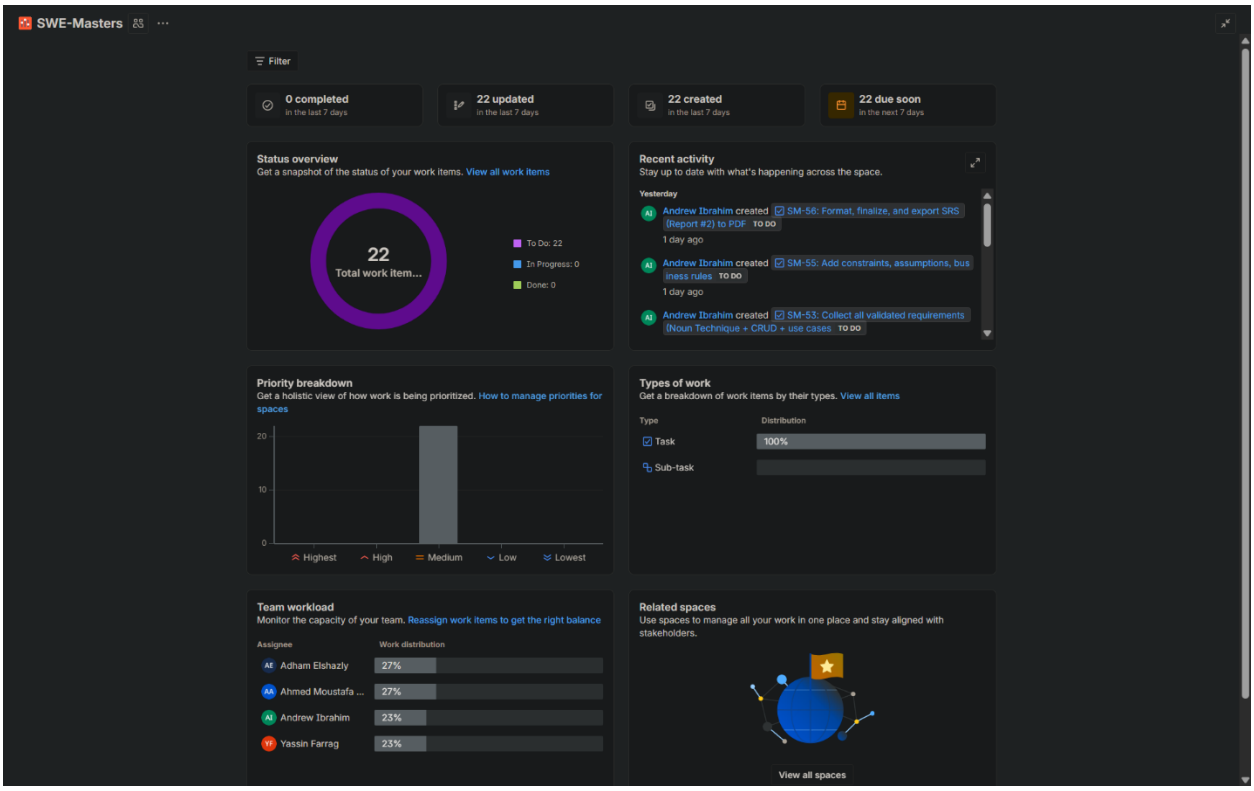
- **System Sequence Diagrams (remaining use cases)**
    - For all other use cases (not covered by Adham):
        - Create SSDs for primary + alternative scenarios.

    - Ensure style and naming are consistent with Member 3's SSDs.
- **SRS (Report #2 – Volere template)**
    - Collect all validated requirements (from Noun Technique, CRUD, use cases).
    - Fill the **Volere-based SRS with Adham**:
        - Functional + non-functional requirements.
        - Constraints, assumptions, business rules.
        - Traceability where needed.
    - Format and polish SRS; export to PDF.

4 of us :

- **Consistency checks**
    - All members review:
        - Class diagram vs use cases vs SSDs vs SRS.
        - Names of classes, use cases, and operations are consistent.
- **Final review meeting**
    - Walk through the entire deliverable together.
    - Fix any conflicts or missing links before submission.

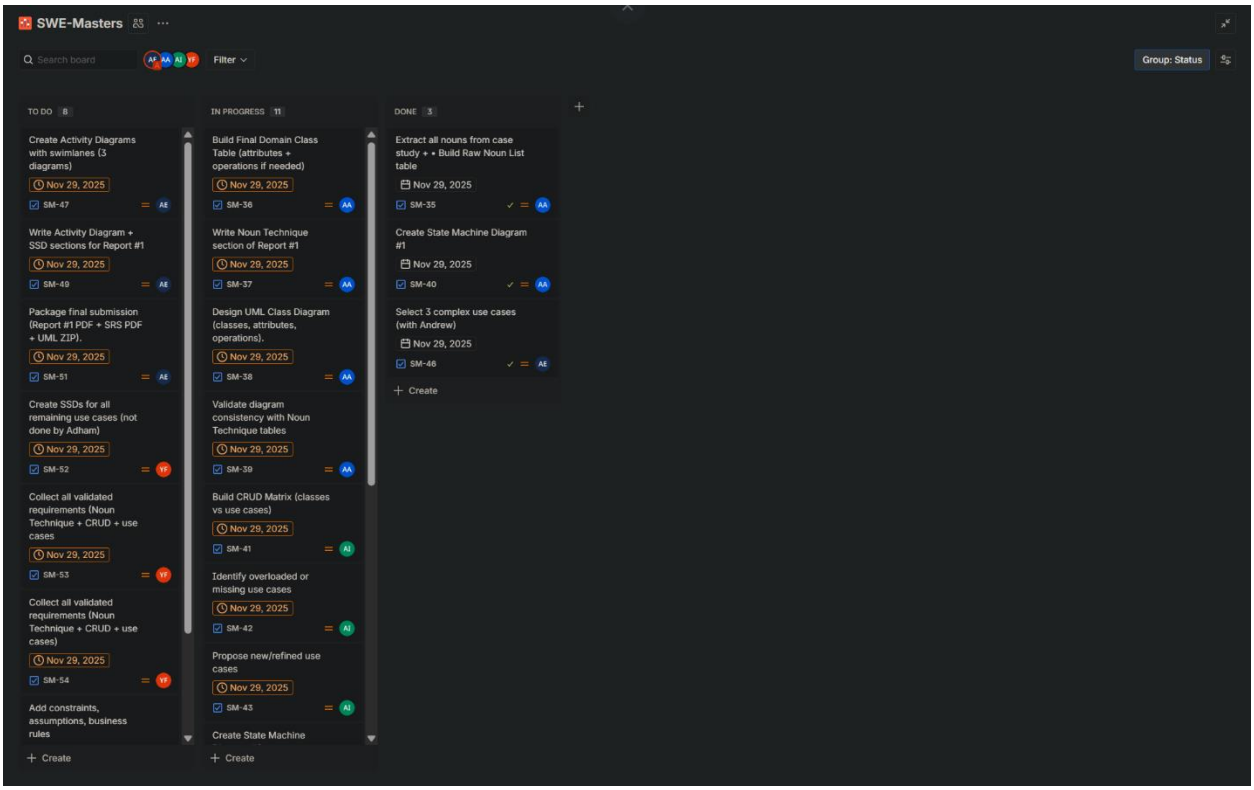
Jira Management



SWE-Masters

Search list | Filter 1 | Group: Category

Type	Key	Summary	Status	Reporter	Comments	Assignee	Due date	Priority	Created	Updated
	SM-35	Extract all nouns from case study + • Build Raw Noun Li...	TO DO	Ahmed Moustafa ...	Add comment	Ahmed Moustafa ...	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-36	Build Final Domain Class Table (attributes + operations if...	TO DO	Ahmed Moustafa ...	Add comment	Ahmed Moustafa ...	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-37	Write Noun Technique section of Report #1	TO DO	Ahmed Moustafa ...	Add comment	Ahmed Moustafa ...	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-38	Design UML Class Diagram (classes, attributes, operatio...	TO DO	Ahmed Moustafa ...	Add comment	Ahmed Moustafa ...	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-39	Validate diagram consistency with Noun Technique tables	TO DO	Ahmed Moustafa ...	Add comment	Ahmed Moustafa ...	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-40	Create State Machine Diagram #1	TO DO	Ahmed Moustafa ...	Add comment	Ahmed Moustafa ...	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-41	Build CRUD Matrix (classes vs use cases)	TO DO	Andrew Ibrahim	Add comment	Andrew Ibrahim	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-42	Identify overloaded or missing use cases	TO DO	Andrew Ibrahim	Add comment	Andrew Ibrahim	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-43	Propose new/refined use cases	TO DO	Andrew Ibrahim	Add comment	Andrew Ibrahim	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-44	Create State Machine Diagram #2	TO DO	Andrew Ibrahim	Add comment	Andrew Ibrahim	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-45	Add explanations for both state machines to Report #1	TO DO	Andrew Ibrahim	Add comment	Andrew Ibrahim	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-46	Select 3 complex use cases (with Andrew)	TO DO	Adham Elshazly	Add comment	Adham Elshazly	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-47	Create Activity Diagrams with swimlanes (3 diagrams)	TO DO	Adham Elshazly	Add comment	Adham Elshazly	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-48	Create SSDs for the 3 selected use cases (primary scen...	TO DO	Adham Elshazly	Add comment	Adham Elshazly	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-49	Write Activity Diagram + SSD sections for Report #1	TO DO	Adham Elshazly	Add comment	Adham Elshazly	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-50	Set up + structure GitHub repository (folders for UML, R...	TO DO	Adham Elshazly	Add comment	Adham Elshazly	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-51	Package final submission (Report #1 PDF + SRS PDF + U...	TO DO	Adham Elshazly	Add comment	Adham Elshazly	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-52	Create SSDs for all remaining use cases (not done by Ad...	TO DO	Yassin Farrag	Add comment	Yassin Farrag	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-53	Collect all validated requirements (Noun Technique + CR...	TO DO	Yassin Farrag	Add comment	Yassin Farrag	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-54	Collect all validated requirements (Noun Technique + CR...	TO DO	Yassin Farrag	Add comment	Yassin Farrag	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-55	Add constraints, assumptions, business rules	TO DO	Yassin Farrag	Add comment	Yassin Farrag	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-56	Format, finalize, and export SRS (Report #2) to PDF	TO DO	Yassin Farrag	Add comment	Yassin Farrag	Nov 29, 2025	Medium	Nov 28, 2025	Nov 28, 2025
	SM-30	Map Events to Use Cases	DONE	Yassin Farrag	Add comment	Adham Elshazly		Medium	Nov 15, 2025	Nov 15, 2025
	SM-31	Verify Actors and System Boundary	DONE	Adham Elshazly	Add comment	Yassin Farrag		Medium	Nov 15, 2025	Nov 15, 2025



SWE-Masters

Search board | Filter | Group: Status

TO DO 8

- Create Activity Diagrams with swimlanes (3 diagrams) — Nov 29, 2025 — SM-47
- Write Activity Diagram + SSD sections for Report #1 — Nov 29, 2025 — SM-49
- Package final submission (Report #1 PDF + SRS PDF + UML ZIP). — Nov 29, 2025 — SM-51
- Create SSDs for all remaining use cases (not done by Adham) — Nov 29, 2025 — SM-52
- Collect all validated requirements (Noun Technique + CRUD + use cases — Nov 29, 2025 — SM-53
- Collect all validated requirements (Noun Technique + CRUD + use cases) — Nov 29, 2025 — SM-54
- Add constraints, assumptions, business rules

+ Create

IN PROGRESS 11

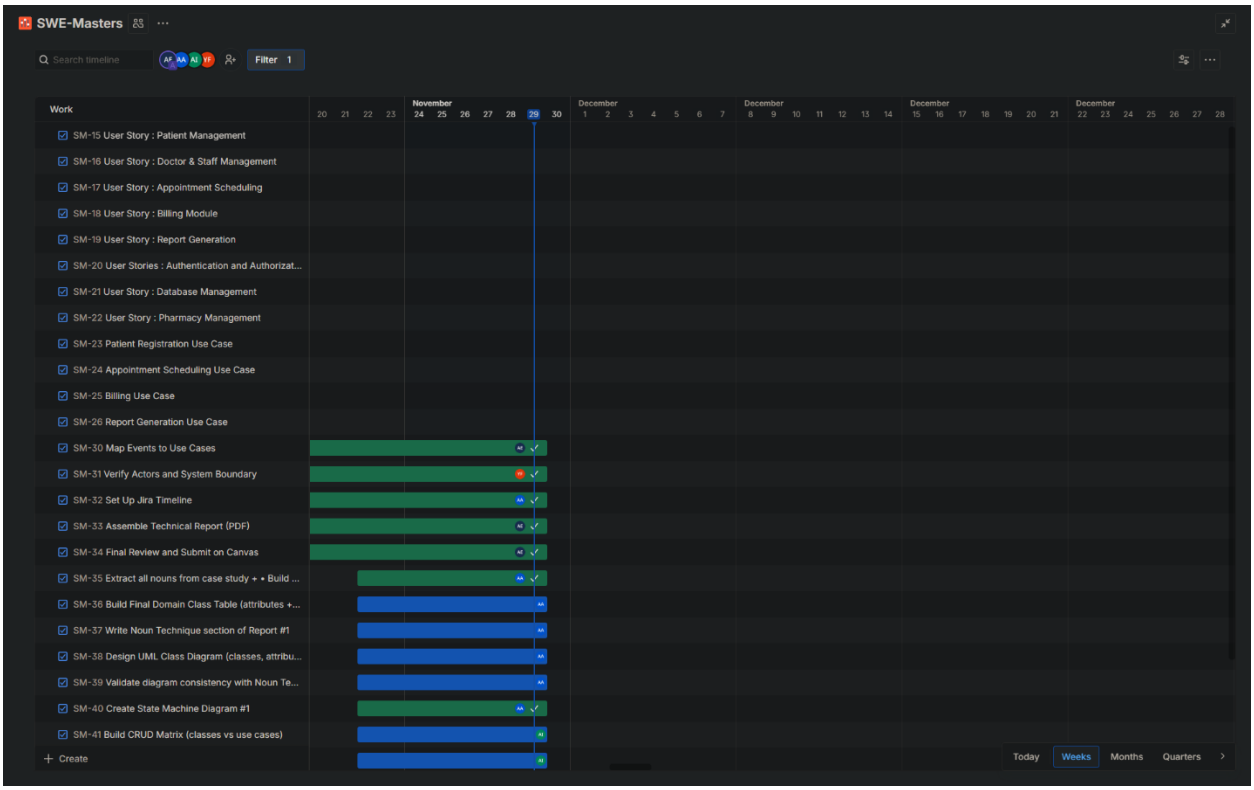
- Build Final Domain Class Table (attributes + operations if needed) — Nov 29, 2025 — SM-36
- Write Noun Technique section of Report #1 — Nov 29, 2025 — SM-37
- Design UML Class Diagram (classes, attributes, operations). — Nov 29, 2025 — SM-38
- Validate diagram consistency with Noun Technique tables — Nov 29, 2025 — SM-39
- Build CRUD Matrix (classes vs use cases) — Nov 29, 2025 — SM-41
- Identify overloaded or missing use cases — Nov 29, 2025 — SM-42
- Propose new/refined use cases — Nov 29, 2025 — SM-43
- Create State Machine

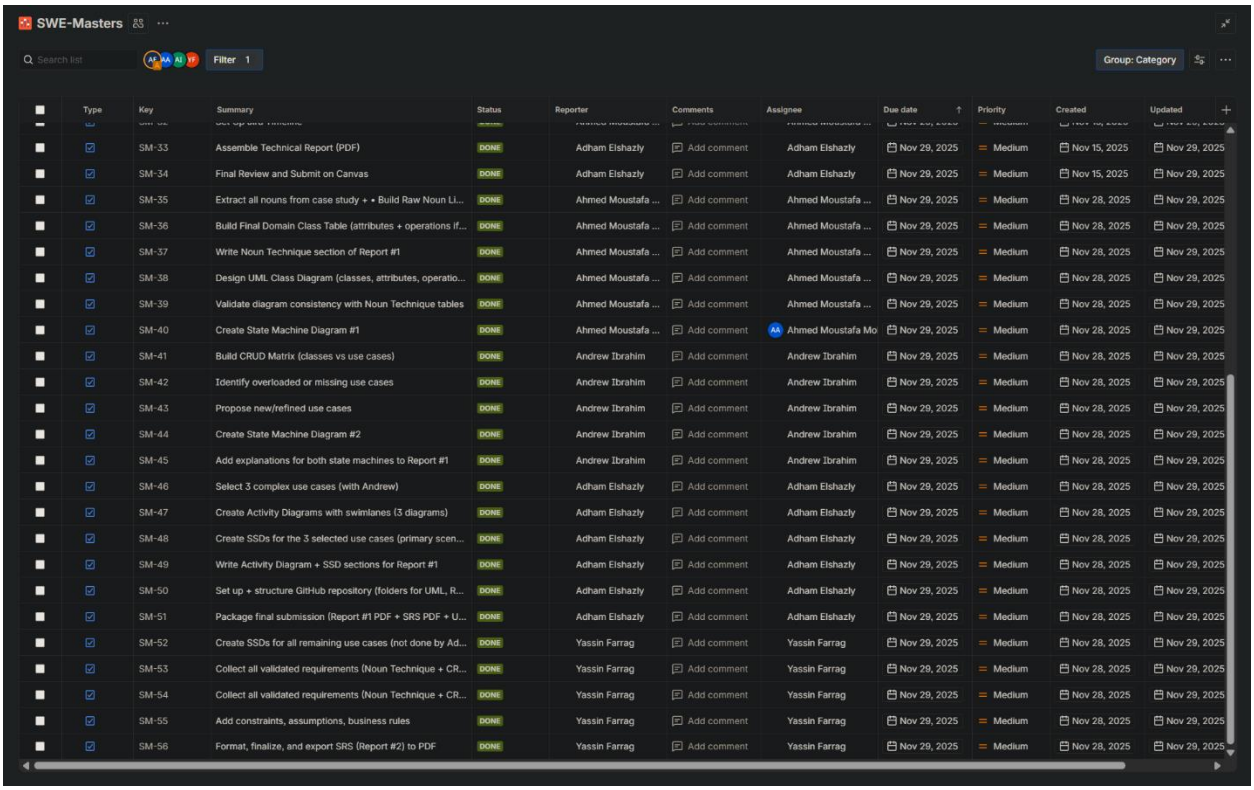
+ Create

DONE 3

- Extract all nouns from case study + • Build Raw Noun List table — Nov 29, 2025 — SM-35
- Create State Machine Diagram #1 — Nov 29, 2025 — SM-40
- Select 3 complex use cases (with Andrew) — Nov 29, 2025 — SM-46

+ Create


SWE-Masters
Search timeline
Filter 1
Work
November
December
SM-15 User Story : Patient Management
SM-16 User Story : Doctor & Staff Management
SM-17 User Story : Appointment Scheduling
SM-18 User Story : Billing Module
SM-19 User Story : Report Generation
SM-20 User Stories : Authentication and Authorizat...
SM-21 User Story : Database Management
SM-22 User Story : Pharmacy Management
SM-23 Patient Registration Use Case
SM-24 Appointment Scheduling Use Case
SM-25 Billing Use Case
SM-26 Report Generation Use Case
SM-30 Map Events to Use Cases
SM-31 Verify Actors and System Boundary
SM-32 Set Up Jira Timeline
SM-33 Assemble Technical Report (PDF)
SM-34 Final Review and Submit on Canvas
SM-35 Extract all nouns from case study + • Build ...
SM-36 Build Final Domain Class Table (attributes +...
SM-37 Write Noun Technique section of Report #1
SM-38 Design UML Class Diagram (classes, attribu...
SM-39 Validate diagram consistency with Noun Te...
SM-40 Create State Machine Diagram #1
SM-41 Build CRUD Matrix (classes vs use cases)
Create
Today
Weeks
Months
Quarters


SWE-Masters
Search list
Filter 1
Group: Category
Type
Key
Summary
Status
Reporter
Comments
Assignee
Due date
Priority
Created
Updated
SM-33 Assemble Technical Report (PDF) DONE Adham Elshazly Add comment Adham Elshazly Nov 29, 2025 Medium Nov 15, 2025 Nov 29, 2025
SM-34 Final Review and Submit on Canvas DONE Adham Elshazly Add comment Adham Elshazly Nov 29, 2025 Medium Nov 15, 2025 Nov 29, 2025
SM-35 Extract all nouns from case study + • Build Raw Noun Li... DONE Ahmed Moustafa ... Add comment Ahmed Moustafa ... Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-36 Build Final Domain Class Table (attributes + operations if... DONE Ahmed Moustafa ... Add comment Ahmed Moustafa ... Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-37 Write Noun Technique section of Report #1 DONE Ahmed Moustafa ... Add comment Ahmed Moustafa ... Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-38 Design UML Class Diagram (classes, attributes, operatio... DONE Ahmed Moustafa ... Add comment Ahmed Moustafa ... Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-39 Validate diagram consistency with Noun Technique tables DONE Ahmed Moustafa ... Add comment Ahmed Moustafa ... Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-40 Create State Machine Diagram #1 DONE Ahmed Moustafa ... Add comment Ahmed Moustafa Mo Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-41 Build CRUD Matrix (classes vs use cases) DONE Andrew Ibrahim Add comment Andrew Ibrahim Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-42 Identify overloaded or missing use cases DONE Andrew Ibrahim Add comment Andrew Ibrahim Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-43 Propose new/refined use cases DONE Andrew Ibrahim Add comment Andrew Ibrahim Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-44 Create State Machine Diagram #2 DONE Andrew Ibrahim Add comment Andrew Ibrahim Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-45 Add explanations for both state machines to Report #1 DONE Andrew Ibrahim Add comment Andrew Ibrahim Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-46 Select 3 complex use cases (with Andrew) DONE Adham Elshazly Add comment Adham Elshazly Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-47 Create Activity Diagrams with swimlanes (3 diagrams) DONE Adham Elshazly Add comment Adham Elshazly Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-48 Create SSDs for the 3 selected use cases (primary scen... DONE Adham Elshazly Add comment Adham Elshazly Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-49 Write Activity Diagram + SSD sections for Report #1 DONE Adham Elshazly Add comment Adham Elshazly Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-50 Set up + structure GitHub repository (folders for UML, R... DONE Adham Elshazly Add comment Adham Elshazly Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-51 Package final submission (Report #1 PDF + SRS PDF + U... DONE Adham Elshazly Add comment Adham Elshazly Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-52 Create SSDs for all remaining use cases (not done by Ad... DONE Yassin Farrag Add comment Yassin Farrag Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-53 Collect all validated requirements (Noun Technique + CR... DONE Yassin Farrag Add comment Yassin Farrag Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-54 Collect all validated requirements (Noun Technique + CR... DONE Yassin Farrag Add comment Yassin Farrag Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-55 Add constraints, assumptions, business rules DONE Yassin Farrag Add comment Yassin Farrag Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025
SM-56 Format, finalize, and export SRS (Report #2) to PDF DONE Yassin Farrag Add comment Yassin Farrag Nov 29, 2025 Medium Nov 28, 2025 Nov 29, 2025